PretextGuard, Explained End to End

Detecting payload-free pretexting emails by checking what an email claims against its headers and its own conversation

Written for Nagasai. 15 October 2026. Matches master document version 3.15.6.

Who this is for. Someone who knows basic machine learning and basic web development, and nothing about email security or research. Every other idea is explained where it first appears and again in the glossary.

How to read this. Parts 1 to 5 build on each other, so read them in order: the project in one page, the background from zero, the core idea, how the system works, and one email and one thread followed through the actual code. Parts 6 to 11 are reference: the novelty, the data, the folder, the technology, the security and how it was built. Part 12 is the evaluation, with every result and what it honestly means. Part 13 is the paper, Part 14 what the system can and cannot do, Part 15 how to explain it to someone, and Part 16 the glossary.

Where the numbers come from. Every number in this document comes from a result file in results/ or from the master document (docs/master_document.md), with its sample stated. Where this document and the master document ever differ, the master document wins.

Contents

1. The project in one page
2. Background you need, from zero
3. The core idea
4. How it works
5. One email and one thread, followed end to end
6. What is new: N1, N2, N3 and the architecture
7. The data
8. A tour of the project folder
9. The technology, one line each
10. Security of the system
11. How it was built
12. The evaluation, and how the numbers were made
13. The paper
14. What it can and cannot do
15. How to explain it to someone
16. Glossary

1. The project in one page

The problem. Some of the most expensive email attacks carry nothing a security scanner can find. There is no link to a fake website and no attachment with a virus. Someone writes as your finance director, your boss or your supplier, tells a believable story and asks you to pay, change bank details or send data. If you believe the story, you do the damage yourself. The FBI's Internet Crime Complaint Center counted 24,768 complaints about this kind of fraud (business email compromise) in 2025, with $3,046,598,558 in reported losses.

Why it is hard. Email security tools are built around the dangerous part of an email: the link or the attachment, which security people call the **payload**. An email with no payload gives them nothing to inspect. Worse, the email often passes every authentication check, because the attacker sends it from a real Gmail account or from a real account they have broken into. The checks say "this really came from gmail.com", which is true, and says nothing about whether the writer is your finance director.

What we built. PretextGuard is a system that reads an email, or a whole email conversation, and does what a careful person would do. It finds what the email claims about itself ("this is David from Finance", "our bank has changed") and checks each claim against the evidence that can confirm or contradict it: the email's own technical headers, and the earlier messages of its conversation. When a claim and its evidence disagree, that disagreement is the finding. It runs as a small website: you paste an email, and you get a risk score from 0 to 100, a band (Low risk, Suspicious, High risk) and a list of findings, each naming the claim, the rule that checked it, the evidence and a reason in plain English.

The one big idea. Never trust what an email says about itself. Authentication proves which domain sent a message. It does not prove who the person is.

The three new things (the "novelties").

- **N1, a fair test.** Before any model reads an email, links, addresses, domains and file names are replaced by placeholders. This shows how much of a normal detector's accuracy comes from reading links, which real pretexting does not have.
- **N2, thread checks.** A reply is compared with the earlier messages of its own conversation. This catches an attacker who is already inside a real account, where every header check passes.
- **N3, claim against headers.** Each claim in the body is checked against what the headers can prove. This catches the outside impersonator.

Around them sits the **claim-routed architecture**: every claim is sent to the one checker that can test it, and every result is recorded as a row in a ledger.

What we found, honestly.

- On made-up takeover replies placed into real conversations, the thread checks caught 18 of 25 Apache cases and 48 of 51 Enron cases. The checks that look at one message alone caught 5 and 0.
- A detector trained on raw email text loses more accuracy when links are hidden than one trained on hidden-link text (a difference of 0.073 in F1, with a 95% interval from 0.061 to 0.086). So part of a normal detector's accuracy is link-reading.
- Every one of the 442 test emails PretextGuard flagged has a stated claim, rule and evidence behind it.

- But as a plain detector over public email collections, it finds fewer attacks than simpler machine-learning systems given the same information. On 3,045 test attacks and 6,357 legitimate test emails its F1 is 0.238, against 0.882 for the best simple combination. The project says so openly.

What it is not. It has never run at a real company. No public collection of real payload-free fraud exists, so the evaluation uses public phishing and fraud collections plus made-up cases. The labels for tactics and claims were made by language models of one family, not by people. It reads English only.

Where it ended. The project produced a working system (Phases 0 to 13), a full evaluation and a 6-page IEEE-style conference paper. Your faculty evaluates the project from that paper, so there is no separate report, viva or review deck.

> **If you remember one sentence.** PretextGuard reads what an email claims about itself and checks each claim against two independent kinds of evidence, its headers and its own conversation history; when the claim and the evidence disagree, that contradiction is the detection.

2. Background you need, from zero

This part teaches everything the rest of the document assumes. It starts with email itself, then the attacks, then the machine learning and the measuring that you may not have met yet, and ends with what a research paper is. If a word later in the document confuses you, it is explained here or in the glossary (Part 16).

2.1 What an email really is

An email is a plain text file. It has two parts separated by one empty line: first the **headers**, then the **body**. The headers are like the envelope and the postmarks: who sent it, to whom, when, and how it travelled. The body is the letter itself. When you download an email from a mail program you get exactly this text in a file ending in **.eml**.

Here is the example the whole project uses. It is made up, but it is a typical attack:

```
From: "David Chen - Finance Director" <d.chen.finance@gmail.com>
Reply-To: finance.dept.acme@protonmail.com
Date: Tue, 14 Jul 2026 02:47 AM
Authentication-Results: spf=pass dkim=pass dmarc=pass (all for gmail.com)

Hi Priya, this is David from Finance. I need you to process a wire
transfer before 3 PM today. I'm in meetings all day and can't take
calls. Please don't loop in anyone else - just get it done.
```

Priya works at a company whose email domain is acmecorp.com. The real David is her company's Finance Director. Keep this email in mind; Part 5 follows it through every step of PretextGuard.

The From line has two parts. The **display name** ("David Chen - Finance Director") is free text that anyone can type. The **address** (d.chen.finance@gmail.com) is where the mail came from, and the part after the @ is the **domain** (gmail.com). Most mail apps, especially on phones, show only the display name. That is exactly what this kind of attack exploits.

A registered domain is the part of a domain that someone actually bought. mail.acmecorp.com and acmecorp.com belong to the same owner, so both have the registered domain acmecorp.com. PretextGuard compares registered domains, using a public list of endings such as .com and .co.uk (the "public suffix list").

The headers that matter most for this project:

Header	What it is	Everyday picture
From	The display name and address shown to the reader	The name printed on the letter
Reply-To	Where replies go, if different from From	"Please reply to this other address"
Return-Path	Where bounces go; the true envelope sender	The return address on the envelope
Received	One line added by every mail server on the way	A postmark from each post office
Authentication-Results	The receiving server's verdicts on SPF, DKIM and DMARC (Section 2.4)	The post office's inspection stamp
Message-ID	A unique ID for this one email	A tracking number

Header	What it is	Everyday picture
In-Reply-To, References	The IDs of earlier emails in the same conversation	"Re: your letter no. 123"
X-Mailer	The mail program that wrote the email	The kind of pen used
List-Id	Present when a mailing list sent the email	A newsletter's masthead

Look at the David email again with these in mind. The display name says Finance Director, but the address is a free Gmail account. The Reply-To quietly sends any answer to a third address at protonmail.com. It was sent at 02:47 at night. None of this is visible on a phone screen.

2.2 How an email travels

When you press Send, your mail program hands the email to your provider's outgoing server. That server looks up which server accepts mail for the recipient's domain (a lookup in the internet's phone book, DNS, for the domain's "MX record") and delivers the email there using the mail protocol, SMTP. The receiving server checks the email, adds its own headers at the top and stores it in the recipient's mailbox.

Two things follow from this that matter later.

- **Every server adds a Received line at the top.** So the Received lines read from bottom to top tell the route, like postmarks. The bottom-most one shows where the email first entered the mail system. PretextGuard calls that first step the **origin** of the email.
- **The envelope and the letter can disagree.** SMTP carries an envelope sender (it ends up in Return-Path) separately from the From line inside the letter. The original protocol checks neither. Anyone can write any From line, which is why authentication (Section 2.4) was invented.

Mailing lists work differently. A list server receives one email and re-sends it to every member. It adds a List-Id header, often sets Reply-To to the list itself, and sends bounces to its own address. So for list mail, a Reply-To that differs from the From, and an envelope that differs from the letter, are perfectly normal. PretextGuard recognises list mail for this reason; the Apache mailing lists in our data are exactly this kind of mail.

2.3 Conversations (threads)

When you reply to an email, your mail program adds two headers to your reply: **In-Reply-To** (the Message-ID of the email you answered) and **References** (the IDs of the whole chain so far). Mail programs use these to group emails into a conversation, called a **thread**. Replies usually also carry the earlier text as a **quotation**, often after a line such as "On Tue, 6 Oct, Ivan wrote:" with each quoted line starting with ">".

Two facts about threads drive a whole part of the project:

- A modern mailing-list archive (like the Apache lists) keeps all these IDs, so its threads can be rebuilt exactly.
- The old Enron email collection (from 2001) has no In-Reply-To or References at all. Its threads can only be guessed, the way old mail programs did it: from subjects ("Re: budget"), dates and shared participants.

2.4 Email authentication: SPF, DKIM and DMARC

Because anyone can write any From line, three checks were added to email over the years. You do not need to know how to set them up, only what each one proves.

- **SPF (Sender Policy Framework).** A domain publishes in DNS the list of mail servers allowed to send for it. The receiving server checks whether the server that delivered the email is on that list. **It proves:** the sending server was allowed to send for that domain.
- **DKIM (DomainKeys Identified Mail).** The sending server signs the email with a secret key. The matching public key is published in DNS, so the receiver can check the signature. **It proves:** that domain signed the email, and nobody changed the signed parts afterwards.
- **DMARC.** Passes when SPF or DKIM passed for a domain that matches the visible From domain (this match is called **alignment**). The domain owner also publishes a policy saying what receivers should do when it fails. **It proves:** the domain in the From line is the domain that passed a check.

The receiving server writes its three verdicts into the **Authentication-Results** header, for example spf=pass dkim=pass dmarc=pass. PretextGuard does not redo these checks; it reads the verdicts the receiving server wrote.

> **The key idea of the whole project.** Authentication proves which domain sent an email, not who the person is. A real Gmail account passes all three checks for gmail.com. So an email that claims to be Acme's Finance Director and authenticates as gmail.com has passed authentication for the wrong domain. Spotting that mismatch is the job of N3.

Three more things to know:

- **Lookalike domains** pass too. Someone who registers paypa1.com (with the digit 1) can set up SPF, DKIM and DMARC perfectly for paypa1.com. Only a comparison with the real domain (paypal.com) shows the trick.
- **An exact-domain spoof** (a forged From line with the real company's own domain) fails DMARC when that company publishes a DMARC policy. Here the verdict itself is the evidence.
- **Not every Authentication-Results header can be trusted.** An attacker can type a fake one into the email they send. Only the ones added by the receiving organisation's own servers count, which sit at the top. The standard that defines this header (RFC 8601) says a receiver must remove incoming copies that pretend to come from inside. PretextGuard reads only the trusted ones (Section 4.2).

2.5 The attacks

Phishing is email that tricks you into giving something away, usually by clicking a link to a fake login page. Most phishing is sent in bulk. **Spear-phishing** is phishing aimed at one person or company. **Spam** is unwanted bulk mail, usually advertising; it is a nuisance, not necessarily an attack. **Advance-fee fraud** (the "Nigerian prince" emails) promises a large sum if you first pay a fee or share your details.

Pretexting is social engineering built on a made-up identity or scenario: "this is the CFO", "this is your supplier". The story itself is the attack, and the victim does the damage personally. Verizon's 2026 Data Breach Investigations Report defines pretexting as building a trusted relationship through invented scenarios, by voice, email or text, with the attacker taking part in the conversation or email thread, and finds it to be the way in for 6% of breaches.

Business email compromise (BEC) is pretexting aimed at company money. Typical forms are a fake executive asking for an urgent wire transfer, a fake or hacked supplier saying "our bank has changed", and a fake employee asking HR to change where their salary is paid.

Payload versus payload-free. A **payload** is the part of an email that does technical damage: a malicious link or attachment. Security scanners are built around payloads: lists of bad links, sandboxes that open links and attachments safely, virus scanners. A **payload-free** email has no link and no attachment, so all of that finds nothing. The David email has no link, no attachment and no typos. The whole attack is three phrases: authority ("this is David from Finance"), urgency ("before 3 PM today") and secrecy ("don't loop in anyone else").

Two attacker positions. This distinction organises the whole project:

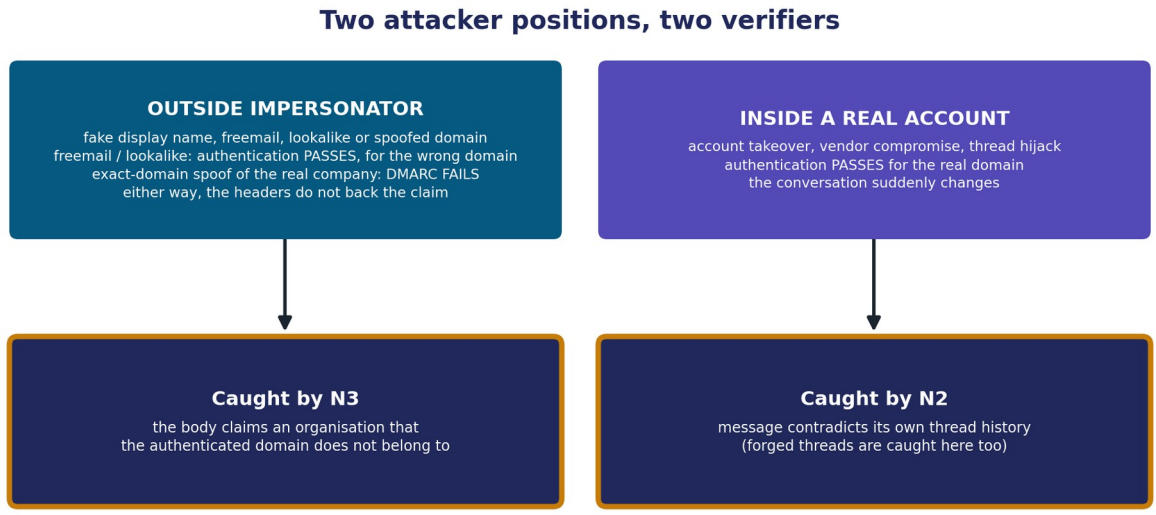


Figure 1. Which checker catches which attacker (docs/figures/figure1_attacker_positions.png)

- **The outside impersonator** writes from a free mailbox (Gmail, Outlook), a lookalike domain or a spoof of the company's own domain, often with a fake display name and a Reply-To that leads elsewhere. The headers do not support who the body says is writing. **N3** catches this.
- **The attacker inside a real account** got there by **account takeover** (they stole the password of a real mailbox), **vendor email compromise** (they took over a supplier's real mailbox) or **thread hijacking** (they reply inside a real, ongoing conversation, so the victim sees an answer in a familiar thread). The email really comes from the real account, so every authentication check passes, and N3 has nothing to contradict. What changes is the conversation: new urgency, new bank details, a new way of sending. **N2** catches this.
- A **forged thread** is a third trick: a new email that includes a fake "earlier conversation" that never happened, so it looks like a reply. N2's thread-integrity check is built for it.

2.6 Persuasion tactics

People who study fraud describe a small set of pressure techniques that scams lean on. The best known list is from the psychologist Robert Cialdini. PretextGuard uses seven **tactics**; one email can use several:

Tactic	What it looks like	Example phrase	Where it comes from
Authority	Claimed rank, role or institutional power	"This is the CFO"; "IT Security requires"	Cialdini

Tactic	What it looks like	Example phrase	Where it comes from
Urgency	A made-up deadline or time pressure	"Before 3 PM today"; "immediately"	Stajano and Wilson's time principle
Scarcity	Loss or finality	"Last chance"; "your account will be deleted"	Cialdini
Reciprocity	Debt	"I covered for you last month, now I need..."	Cialdini
Social proof	Everyone else already did it	"The whole team has already signed off"	Cialdini
Liking	False warmth, flattery, familiarity	"Great seeing you at the offsite, quick favour..."	Cialdini
Isolation or secrecy	Cutting you off from checking	"Keep this between us"; "don't loop in your manager"	Business email compromise reports

Secrecy matters most: a real business request almost never asks you to avoid checking it. But a tactic on its own is weak evidence. Real work email is often urgent, and real managers say "please do this today". One study the paper cites found authority cues in 27.10% of legitimate email samples. That is why PretextGuard never raises an alarm because of a tactic alone: a tactic only makes a contradiction that was already found weigh more (Part 4).

2.7 The machine learning you may not have met

You know the basic idea: a model learns from labelled examples, is checked on examples it has not seen, and makes predictions. Here is what this project adds to that.

Text has to become numbers. A model works on numbers, so text is turned into numbers first. Two ways are used here.

- **TF-IDF.** Count how often each word appears in an email, and weight each count by how rare that word is across all emails, so that "the" counts for little and "wire" counts for more. Each email becomes a long list of numbers, one per word in the vocabulary. A **logistic regression** then learns one weight per word and adds them up into a probability. This pair is cheap, fast and surprisingly strong, which is why it is a standard comparison (a **baseline**).
- **Transformers.** A transformer reads a sentence as a sequence of **tokens** (words or pieces of words) and works out what each token means in its context, so "bank" in "river bank" and "bank account" end up with different numbers. **BERT** is a famous transformer that was **pretrained** by reading a huge amount of English with a simple game (guess the hidden word), which taught it a lot about language before it ever saw an email.

DistilBERT is a smaller, faster version of BERT: 6 layers and about 66 million learned numbers (its authors report it is about 40% smaller and 60% faster while keeping about 97% of BERT's language understanding). It is small enough to train on Google's free Colab GPU and to run on a laptop CPU in a fraction of a second per email.

Fine-tuning means taking a pretrained model and training it a little more on your own labelled examples, so it learns your task on top of what it already knows. PretextGuard fine-tunes DistilBERT to recognise the seven tactics.

Multi-label output. An email can use several tactics at once, so the model gives seven separate probabilities, one per tactic (for example urgency 0.94, liking 0.03), instead of picking one class. A tactic "fires" when its probability is above its **threshold**. A threshold of 0.5 is only a default;

with rare tactics the probabilities are shifted, so each main tactic gets its own threshold, chosen on the validation emails (Section 2.8).

Seeds. Training involves randomness (the starting numbers, the order of examples). A **seed** fixes that randomness so a run can be repeated. Training with several seeds shows how much results move just by chance.

Rules versus learning. Not everything needs a trained model. A rule written by hand ("if the text says 'this is X from Y', that is an affiliation claim") needs no training data, is fast, and can always say why it fired. Its weakness is that it only knows the phrasings someone wrote down. PretextGuard uses rules for claims and checks, and a learned model only for tactics.

Named-entity recognition. A tool that marks the names of people and organisations in text. PretextGuard uses **spaCy**, a common language-processing library, for this and to cut text into tokens. Patterns are then written over tokens, not over raw characters. This matters for safety: a carelessly written pattern over characters (a "regular expression") can take hours on a cleverly crafted input, an attack called **ReDoS**; token patterns cannot.

LIME. A way to ask a model "which words made you say urgency?". It makes a few hundred copies of the email with random words hidden, watches how the urgency probability changes, and fits a small straight-line model to those changes. The words with the largest weights are the ones that pushed urgency up, and PretextGuard highlights them on screen. LIME explains the learned model only; the checks explain themselves through their written reasons.

Labels made by language models. To train and test the tactic model you need emails labelled with their tactics. Nobody had labelled ours, so two large language models (from Google's Gemini family) labelled the same emails separately, and a third broke ties where they disagreed. How far two labellers agree beyond what chance would give is measured with **Cohen's kappa**: 0 means no better than chance, 1 means perfect agreement, and around 0.4 to 0.6 is usually read as moderate. Agreement between two models of one family is not the same as being right, which is a limit the project states everywhere.

No language model at run time. The language models were used only once, offline, to make labels and some test emails. The running system uses no language model at all: it is DistilBERT plus rules. That keeps it fast, free, repeatable and explainable.

2.8 How results are measured

The confusion matrix. For a detector that flags emails as attacks, every email lands in one of four boxes:

	Really an attack	**Really legitimate**
Flagged	True positive (TP): caught	False positive (FP): a false alarm
Not flagged	False negative (FN): missed	True negative (TN): correctly left alone

From these come the measures used everywhere in this document:

- **Precision** = TP / (TP + FP). Of the emails flagged, how many were attacks.
- **Recall** (also called the detection rate) = TP / (TP + FN). Of the attacks, how many were flagged.
- **F1** combines the two into one number between 0 and 1 (their harmonic mean). It is high only when both are high.
- **Macro-F1** is the plain average of the F1 of several classes (for example four tactics), so a rare tactic counts as much as a common one.

- **False alarm rate** (false positive rate) = FP / (FP + TN), usually measured on legitimate mail ("ham"). It matters a great deal in security: a tool that cries wolf gets switched off.
- **AUC** (area under the ROC curve) measures ranking, without choosing any cut. It is the chance that a randomly chosen attack gets a higher score than a randomly chosen legitimate email. 0.5 is a coin toss; 1.0 is perfect separation.

Train, validation and test. The data are split three ways before anything is built. The **train** split is for learning and for writing rules. The **validation** split is for choosing settings: which training round to keep, which thresholds to use. The **test** split is touched once, at the very end, for the final numbers. If you choose anything after looking at the test split, the test score stops being honest, because you have tuned to it. That is called **leakage** or **data snooping**, and Part 12 shows how the project guarded against it.

Grouped splitting. Emails from one conversation or one spam campaign look alike. If some go to train and some to test, the test becomes easy for the wrong reason. So emails sharing a subject are kept together in one split, and so are the messages of one thread.

Confidence intervals and the bootstrap. Every result is measured on a limited sample, so it would come out a little different on another sample. A **95% interval** shows that range: F1 0.238 [0.204, 0.277] means the measured value is 0.238 and the plausible range is 0.204 to 0.277. The project gets intervals by the **bootstrap**: draw a new sample of the same size from the test set with replacement, recompute the number, repeat 1,000 times, and take the middle 95% of the results. It draws whole groups (whole subject groups, whole threads, whole synthetic pairs), because emails inside a group are not independent.

Paired differences. To compare two systems, the difference is computed on the same resampled emails each time, which gives an interval for the difference itself. **If that interval includes 0, the two systems cannot be told apart, and neither may be called better.** This rule is used throughout.

Small cells are counts. When fewer than 10 positive examples exist, a rate jumps around with every single email, so the project reports the raw counts instead (for example "1 found, 2 false alarms, 3 missed").

Baselines and ablations. A **baseline** is the simple thing you compare against, so your number means something. An **ablation** removes or swaps one component and measures again; the difference shows what that component contributed. Each novelty of this project has its own ablation.

Confounds and shortcuts. A model takes the cheapest route to a correct answer. If all the attacks in your data come from one collection and all the legitimate mail from another, a model can learn to recognise the collection (its writing style, its era, its headers) instead of the attack. That is a **confound**, and the cheap route is a **shortcut**. Two shortcuts matter here: links (phishing has links, so a model can learn "link means attack", which fails on payload-free fraud) and collections (each source of our data has its own style). The project measures both rather than hoping they are absent.

2.9 What a research paper is

A **research paper** makes claims and backs each one with evidence: results the authors measured, or earlier work they cite. A **conference paper** is a short paper (here 6 pages, two columns, IEEE style) submitted to a conference, where other researchers judge it before it is accepted (**peer review**); in **double-blind** review neither side knows who the other is.

- **Novelty** or **contribution** is what the paper adds that did not exist before. Stated precisely it is the paper's strength; stated loosely it is the easiest thing for a reviewer to knock down.
- **Related work** explains what others already did and where the new work fits. Every cited work must have been read, at least at the level the paper relies on.
- **A limitation** is something the work does not do or does not show, stated openly. Reviewers trust a paper more when it names its own limits.
- **A threat to validity** is a reason a result might not mean what it seems to mean, such as the collection confound above.
- **Honest reporting** means an unwelcome result is reported as it is, and the system is not changed after seeing the test results to make the number better. PretextGuard's evaluation has an unwelcome central result, and Part 12 shows how it is reported.

3. The core idea

3.1 The immigration officer

Picture an immigration officer at an airport. A traveller walks up and says: "I am a citizen of this country, I work for the Finance Ministry, and I am returning from a business trip." The officer does not judge the traveller by how confident they sound. The officer takes each statement and checks it against the document that can prove it.

- "I am a citizen" is checked against the passport: which country issued it?
- "I work for the Finance Ministry" is checked against the official ID or the visa category.
- "I am returning from a trip" is checked against the travel history: did this person actually leave?

A genuine passport from another country is a real document, properly issued, and it still contradicts "I am a citizen of this country". The document is valid; the claim is false.

PretextGuard is that officer, for email:

At the airport	In PretextGuard
The traveller's statements	The claims in the email body ("this is David from Finance")
The passport and its issuing country	The authenticated domain in the headers (who SPF, DKIM and DMARC vouched for)
A real passport from the wrong country	A real Gmail account claiming to be Acme's finance director
A forged passport	A spoofed From line that fails DMARC
A passport from a country with a near-identical name	A lookalike domain such as paypa1.com
The travel history	The earlier messages of the thread
A frequent traveller whose story suddenly changes	A hijacked account whose new message asks for a new bank account
The officer's written note: what was claimed, what was checked, the verdict	The ledger row: claim, rule, evidence, reason
Nervousness, hurry, "please let me through quickly"	Tactics (urgency, secrecy): they make the officer look harder at a problem already found, but are not a reason on their own

The last row is the subtle part. A nervous traveller with a perfect passport is let through. A nervous traveller with the wrong passport is a serious case. Pressure makes a contradiction worse; it never creates one. PretextGuard's score works exactly this way.

3.2 Why "conditioning on the claim" matters

The same header values can mean opposite things depending on what the email claims. That is the heart of N3, so here are two pairs:

- dmarc=pass for gmail.com on a message from a friend is normal. dmarc=pass for gmail.com on a message that claims to be Acme's Finance Director is a **contradiction**: authentication passed for the wrong domain.

- spf=fail on a newsletter is mildly odd and low risk. spf=fail on a message that claims to be the CFO and orders a payment is high risk.

Nothing changed in the headers between the two halves of each pair. Only the claim changed. A system that turns the headers into a score on their own ("SPF passed, good") cannot express "the body claims X and the headers disprove X", because no part of it knows which claim the header evidence is about. PretextGuard routes each claim to the check that can test it, so every check knows what it is checking.

3.3 Three kinds of answer, not two

When a claim is checked, there are three possible results, not two:

- **Contradiction**: the evidence disagrees with the claim (with a severity: high, medium or low).
- **Consistent**: the evidence fits the claim, or at least nothing contradicts it.
- **Not checkable**: the evidence the check needs is missing. For example, a pasted email without headers cannot be checked against its headers, and "this is David from Finance" cannot be checked if we do not know which company the recipient works for.

"Not checkable" is never treated as "fine". Missing evidence is neither a contradiction nor proof of honesty, and the report says how many claims could not be checked. This one rule prevents a whole class of silent mistakes.

3.4 What the system never does

- It never trusts what an email says about itself, including a fake Authentication-Results header written by the sender.
- It never raises an alarm because of tactics alone.
- It never stores the emails it analyses: each email lives in memory for one request and is then discarded.
- It never calls a language model while running.

4. How it works

4.1 The whole picture

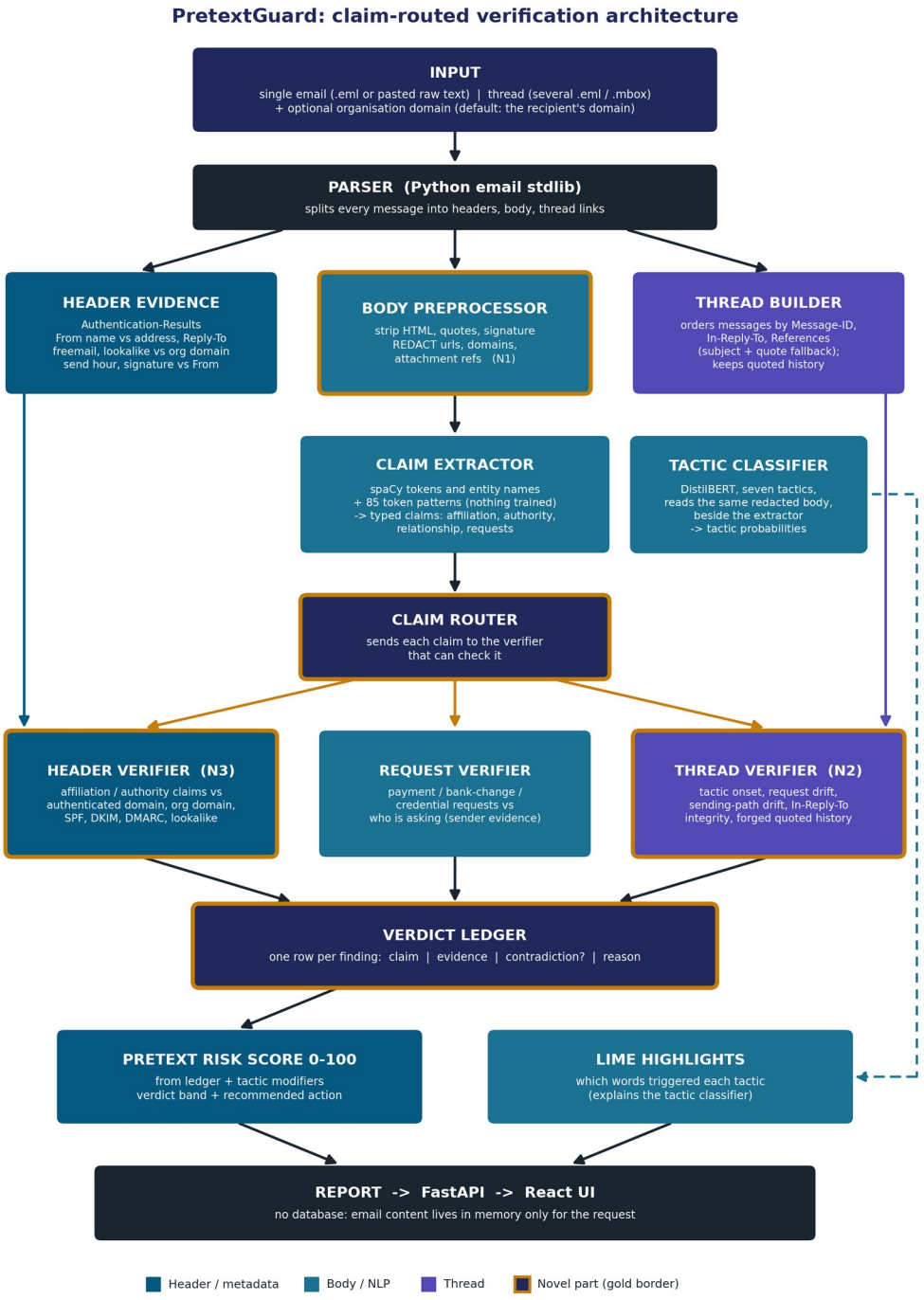


Figure 2. The claim-routed verification architecture; gold borders mark the parts that are new (docs/figures/figure2_architecture.png)

An email (or a thread, plus optionally the recipient's company domain) comes in at the top. A **parser** splits it into headers, body and thread links. Three branches follow: the **header evidence** branch, the **body** branch (cleaning and redaction) and, for a thread, the **thread builder**. From the body, a **claim extractor** finds typed claims and, beside it, the **tactic classifier** gives tactic probabilities. The **router** sends each claim to the checker that can test it: the **header verifier** (N3), the **request verifier** or the **thread verifier** (N2). Every checker writes rows into the **verdict**

ledger, the ledger produces the **risk score**, and **LIME** highlights the words behind each tactic. The report goes back through the API to the web page. Nothing is stored.

The rest of this part takes each box in order. For each one: what it does, how, and why it was built that way.

4.2 The parser and the header evidence

What it does. Turns the raw email text into separate header fields, and then into a tidy set of facts about the sender: the evidence.

How. Python's built-in email package, with its most forgiving parser, because spam and phishing often have broken headers. Each header field is parsed on its own, so one broken field costs only itself. Python's strict address parser gives up on From lines such as service@paypal.com <x@evil.ru> (a display name that is itself an address, a common trick). When it gives up, PretextGuard takes the last address in angle brackets, because that is where replies actually go. With this fallback, the share of phishing_pot From lines that could be read rose from 72.0% to 92.1%.

The evidence it produces (all in src/headers):

- The **SPF, DKIM and DMARC verdicts**, read only from **trusted** Authentication-Results headers: the topmost one, plus those directly below it from the same organisation, stopping at the first one from anyone else. Both the standard format and Microsoft's slightly different one are read. A missing verdict is recorded as **unknown**, never as pass.
- The **authenticated domain**: the domain a passing check vouched for (from DMARC, else DKIM, else SPF). For the fake David it is gmail.com.
- The **From** name, address and domain; whether the display name itself shows an address.
- **Freemail**: whether the sender uses a free mailbox provider (a hand-written list of about 100, plus open platforms where anyone can create a sub-domain, such as onmicrosoft.com).
- The **lookalike score**: how similar the sender's domain is to the company's domain, from 0 to 100, after undoing common tricks (the digit 0 for the letter o, 1 for l, "rn" for "m", Cyrillic letters that look Latin). paypa1.com scores 100 against paypal.com.
- **Reply-To divergence** (replies go to another domain), **envelope mismatch**, the Received hops and the **origin** server, the send hour, whether it is **list mail**.
- The **organisation domain**: the recipient's company domain, taken from the To address unless the user supplies it. It is left empty when the To address says nothing about a company (a Gmail recipient, a research collection's mailbox, a placeholder such as example.com). Then internal-affiliation checks are "not checkable".

Why. Every header is written by the sender except the ones the receiving side adds, so the parser treats headers as hostile input: sizes are capped, fields are parsed separately and fake authentication headers are ignored.

4.3 Cleaning and redaction (the body branch, N1)

What it does. Produces three versions of the body: **clean** (readable text, links kept), **redacted** (links and addresses replaced) and the **signature** block.

How. HTML is turned into text (with BeautifulSoup); mailing-list footers and quoted history are removed; the signature (after "-- " or a closing like "Best regards,") is copied out. Then redaction replaces, in this order:

What	Becomes	Example
A web address	[URL]	https://paypa1.com/login
An email address	[EMAIL]	d.chen.finance@gmail.com
An attachment file name	[FILE]	invoice.zip
A bare domain	[DOMAIN]	acmecorp.com

Words such as "see attached" stay, because they are language, not payload. A domain is only replaced when it ends in a real ending from the public suffix list, so "Mr.Smith" and "e.g" survive.

Why. Real pretexting has no link. If the models read links, they learn "link means attack", which is useless against the attacks this project is about (Part 6, N1). So every model and every rule reads the redacted text. The clean text with links exists only as "model A's view" in the N1 experiment.

4.4 The claim extractor

What it does. Reads the redacted body and the signature and lists the **claims**: statements the email makes about itself. There are eleven claim types:

Claim type	Example	Checked by
affiliation_internal	"This is David from Finance" (claims to be from the recipient's own company)	Header verifier
affiliation_external	"PayPal Security Team", "Central Bank"	Header verifier
authority	"As CFO I need this today"	Header verifier
reply_direction	"Reply to my private email"	Header verifier
signature_contact	A phone number or address in the signature	Header verifier
prior_relationship	"As we discussed on the call"	Thread verifier
payment_request	"Process a wire"	Request verifier (+ thread verifier)
payment_change	"Use our new bank account"	Request verifier (+ thread verifier)
credential_request	"Confirm your password"	Request verifier (+ thread verifier)
gift_card	"Buy five gift cards"	Request verifier (+ thread verifier)
data_request	"Send me the employee list"	Request verifier (+ thread verifier)

How. No model is trained and no language model runs. spaCy cuts the text into tokens and marks names of people and organisations. Then 85 hand-written token patterns (for phrasings like "this is [name] from [department]"), an organisation rule (an organisation name next to a word like Team, Bank or Security) and a signature rule turn the text into claims. Each claim records its type, its exact text, where it sits in the text, which pattern found it, and a **confidence** of 0.9 (a strong pattern, hard to say innocently) or 0.6 (a weak one, common in ordinary mail). The confidence is the strength of the rule, not a probability.

Why rules. The claim types are a closed list of typical phrasings, so rules need no training data beyond a few hundred labels for testing. Every claim can say which rule produced it. The weakness is paraphrase: an attacker who words a claim in a way no pattern expects is missed. The evaluation measures how big that weakness is (Part 12).

The patterns were written by reading the train split only, in four versions (0.1 to 0.4); version 0.4 was frozen before the validation emails were scored once.

4.5 The tactic classifier

What it does. Gives seven probabilities, one per tactic, for the redacted body (the first 2,000 characters).

How. DistilBERT fine-tuned on Colab on 413 labelled real training emails plus 306 synthetic ones, three seeds, keeping the best. Thresholds were chosen on validation emails: authority 0.55, urgency 0.45, scarcity 0.65, secrecy 0.90, and 0.50 for the three rare tactics. On a laptop it runs in a fraction of a second per email.

Its role is deliberately small. Tactics are never checked by any verifier, because no header can prove or disprove urgency. Only four tactics (authority, urgency, scarcity, secrecy) count in the score, and only a little (Section 4.9). Reciprocity, social proof and liking are shown but not scored, because there were too few real examples to test them.

4.6 The router

What it does. Sends every claim to the checker or checkers that can test it, using a fixed table (the "Checked by" column above). With a thread, the five request types also go to the thread verifier, and a prior-relationship claim can only be checked there. Tactics go nowhere.

How. A lookup table in src/verifiers/rows.py and a small function in src/router/router.py. If you know Express, it is a route table: a claim type is the path, a verifier is the handler. A claim of an unknown type is reported as skipped, never dropped silently, and a check makes sure every claim got a row and every row names a claim.

Why. This is the "claim-routed" in the architecture's name. Because each check receives one claim, it knows what it is checking and can write a reason about that claim.

4.7 The verifiers

There are three. Every rule has a name (starting hv_ in the header verifier, rv_ in the request verifier, tv_ in the thread verifier), and every rule writes a ledger row. The rules of the header and request verifiers were frozen at version 0.3, the 44 thread rules at version 0.2.

The header verifier (N3) compares three domains: the **From** domain the reader sees, the **authenticated** domain the checks vouched for, and the **claimed** domain (the company's own domain for an internal claim, or a known brand's domains for an external claim; 51 brands are listed in src/verifiers/brands.py). Examples of contradictions:

- Claims to be internal, but sends from a free mailbox (hv_int_freemail, high). This is the fake David.
- Claims to be internal, uses the company's own domain in From, but fails DMARC: an exact-domain spoof.
- Claims to be PayPal, but sends from a domain that is not PayPal's, or from a lookalike.
- Says "reply to me directly", and Reply-To points to another domain.
- The signature gives a different address from the one the email was sent from.

The request verifier asks who is asking for the money, the password or the data. It looks at seven signals about the asker: a lookalike domain, a failed DMARC check, a free mailbox, a Reply-To elsewhere, a misleading display name, a failed SPF check, and no check passing at all. It also finds bank details without regular expressions and checks IBAN numbers with their

built-in checksum; account numbers appear masked (country and last four characters). A request from a free mailbox (rv_freemail) is the core business email compromise rule.

The thread verifier (N2) compares the newest message with the messages before it on four signals:

Signal	What is compared	Attack it catches
Tactic onset	Does a tactic suddenly cross its threshold, well above its earlier average?	Weeks of calm invoice emails, then urgency and secrecy
Request drift	Payment details, bank accounts, credential or data requests that never appeared before	"We changed banks, please use this new account"
Sending-path drift	The origin server, the mail program, the sender's domain, against the same sender's earlier messages	Same address, but a new server and a new mail program; or the sender's name at a lookalike domain
Thread integrity	Do In-Reply-To and References point to real earlier messages, and does the quoted text match what was really said?	A forged "earlier conversation"

Quotations are compared as sets of five-word runs, so re-wrapped lines still match. The thread verifier also scans the whole thread and reports the first message with a medium or high contradiction: **the point where the thread flipped**.

One change alone is weak evidence: people get new phones and travel, so a new origin server on its own is low severity, and a rule that turned out to fire often on real threads (tv_path_origin) counts zero in the score.

4.8 The verdict ledger

Every verifier writes rows in one fixed shape. Here is the row for the fake David:

```
claim:     affiliation_internal "this is David from Finance"
verifier:  header
rule:      hv_int_freemail
evidence:  from_domain gmail.com, org_domain acmecorp.com, freemail true,
           spf pass, dkim pass, dmarc pass
verdict:   contradiction, severity high
reason:    Claims to be internal (Finance at acmecorp.com), but the message
           comes from gmail.com, a free mailbox provider anyone can use.
           Authentication passed for gmail.com.
```

The ledger is the explanation. Nothing is explained "afterwards": the reason was written by the rule that found the problem, from fixed sentence templates and cleaned values, so it can never contain text an attacker controls in a dangerous form. Severity is the strength of the rule, not a probability.

4.9 The risk score

The score turns the ledger into one number from 0 to 100. The rules, as frozen (score version 0.2):

1. **Only contradictions add points.** Consistent rows add nothing and take nothing away. Not-checkable rows add nothing.

2. **Points per severity:** high 60, medium 35, low 5.
3. **Reliability.** A rule that turned out to fire often on legitimate mail in the train split counts half or zero. Three rules count zero: hv_sig_freemail_sender, hv_sig_other_domain and tv_path_origin.
4. **One claim counts once:** only the strongest row of each claim counts.
5. **Diminishing returns:** sort the claims by points; the second counts half, the third a quarter, and so on. A second independent finding is strong evidence; a tenth adds almost nothing.
6. **Pressure multiplies:** if urgency or secrecy fired, the contradiction points are multiplied by 1 plus 0.25 for each.
7. **Small tactic points:** authority, urgency, scarcity and secrecy add 4 points each when they fire, at most 12. On their own they cannot reach the Suspicious band.
8. **Cap** at 100.

Bands: 0 to 34 **Low risk**, 35 to 69 **Suspicious**, 70 to 100 **High risk**. "Low risk" means no contradiction was found among the claims that could be checked, never "safe". Each band comes with a fixed recommended action, such as "confirm through a known phone number before acting".

Worked values: a lone medium contradiction scores 35 (Suspicious), a lone high 60 (Suspicious), a lone high with urgency 75 (High risk), two highs 90 (High risk), twelve lows about 10 (Low risk).

How the numbers were chosen without labels. Nobody labelled contradictions, so the score could not be fitted to "attack or not". Instead, the meaning above was fixed by hand, and a **false-alarm budget** was declared before the validation emails were read: for each source of legitimate mail, at most 1% High risk and at most 5% Suspicious or above, among emails that had a checked claim (judged only where at least 20 such emails exist). A small grid of 27 alternatives was then tried on validation. None kept the meaning and met the budget everywhere, so the rule said to keep the initial numbers (60, 35, 0.25) and report the miss. That is what happened.

4.10 LIME highlights

For each tactic that fired, LIME (Section 2.7) finds the words that pushed it up, and the page highlights them. It is written by hand in src/explain/lime_explain.py (300 copies of the email with random words hidden, a fixed seed). A deletion test on 33 explanations from real validation emails checked that the highlighted words matter: removing them lowered the tactic probability by 0.269 on average, against -0.010 for random words of the same frequency, and the tactic stopped firing in 67% of cases against 6%. Two honest caveats: many highlighted words are small function words ("I", "to"), because LIME shows what the model reacts to, which is not always what a person would underline; and the exact words change with the random seed. LIME costs about as much as analysing 300 emails, so the API offers it as a separate, rate-limited call.

4.11 The API and the website

The API (src/api, FastAPI) has five routes:

Route	What it does
POST /analyze	Analyses one email, without highlights
POST /analyze/thread	Analyses a thread; the newest message is judged against the others
POST /explain	The same analysis plus LIME highlights; a smaller rate limit
GET /results	The evaluation tables the dashboard shows, from a fixed list

Route	What it does
GET /health	Whether the model is loaded

The POST routes need a secret key in an X-API-Key header. The whole security design is in Part 10.

The website (frontend/, React with Vite and Tailwind) has two pages. The **analyzer** takes a pasted email, a .eml file or several files for a thread, and shows the score and band, the findings table, the text the system read with the claims and tactic words highlighted, the tactics, how the score was built, the header findings, and for a thread a timeline that marks the flip point. The **dashboard** shows four charts and every result table of the evaluation. The page talks to the API through a small proxy that adds the key, so the key never reaches the browser.

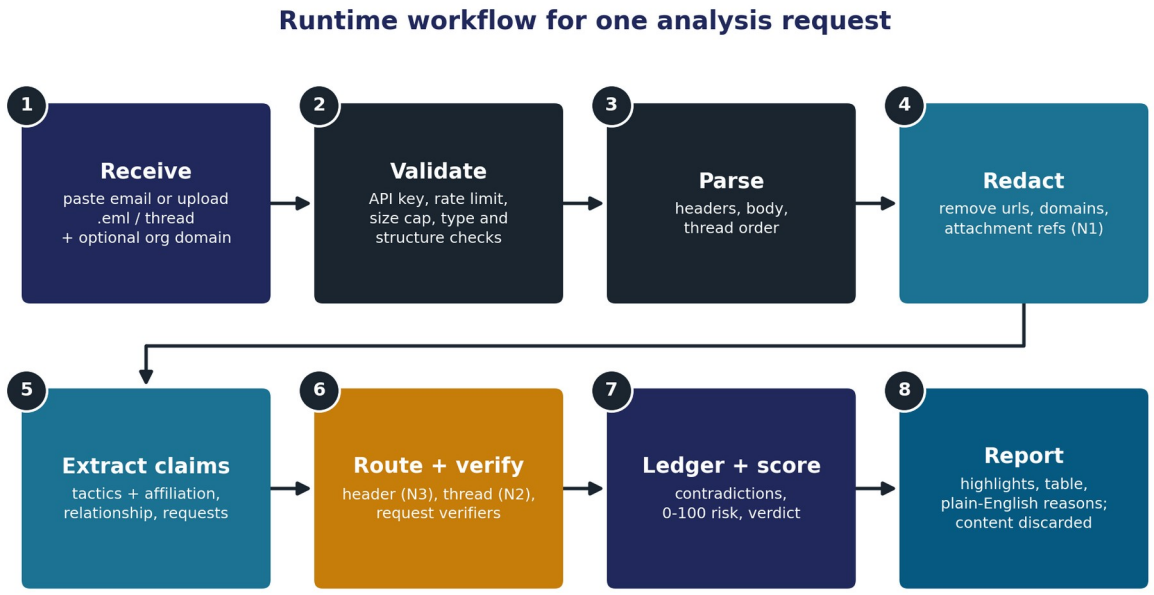


Figure 3. The eight steps of one request (docs/figures/figure3_runtime_workflow.png)

4.12 Build time and run time

The project has two halves. **Build time** happened once, during Phases 1 to 13, on the Mac and on Colab: collect and clean the data, label it, train the tactic classifier, run the experiments. **Run time** is the web app: it analyses one request at a time and stores nothing. The only thing that crosses from build time to run time is the trained model in artifacts/ (about 270 MB, kept only on the Mac and not in GitHub). The experiment numbers feed the paper and the dashboard.

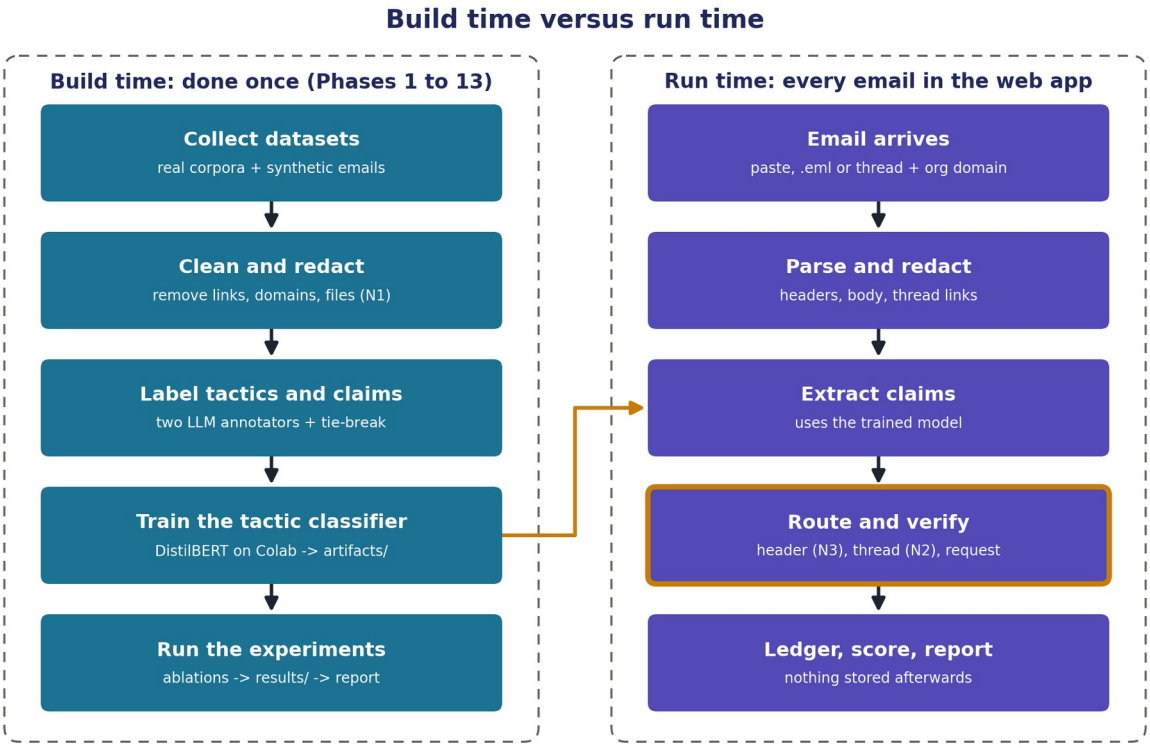


Figure 4. Build time versus run time (docs/figures/figure5_build_vs_run.png)

5. One email and one thread, followed end to end

5.1 The fake David, step by step

Priya pastes the David email (Section 2.1) into the analyzer page and enters her company's domain, acmecorp.com. (If she leaves the domain empty, it is taken from the To address.) Here is everything that happens, with the file where each step lives.

1. **The browser sends it.** frontend/src/components/EmailInput.jsx holds the form. frontend/src/lib/useAnalysis.js runs the two-call flow: first POST /api/analyze, then POST /api/explain for the highlights. frontend/src/api.js is the only file that touches the network. The request goes to the small server that serves the page (configured in frontend/vite.config.js), which adds the secret API key and forwards it to the API.
2. **The API checks the request before reading it.** src/api/main.py receives POST /analyze. Before the body is even read, the rate limit is counted and the key is compared (src/api/security.py). Then the body's size is counted as it arrives, and its shape is checked against a strict schema (src/api/schemas.py). Only then does the API call analyze(), one analysis at a time.
3. **Parse and gather evidence.** src/router/pipeline.py, function analyze(), is the heart of the run. It splits the headers from the body (src/headers/parser.py) and builds the evidence (src/headers/evidence.py): From domain gmail.com, freemail yes, authenticated domain gmail.com, SPF, DKIM and DMARC pass, Reply-To at protonmail.com, organisation domain acmecorp.com.
4. **Clean and redact.** src/preprocess/clean.py and redact.py. This body has no links, so the redacted text is the same as the clean text.
5. **Tactics.** src/models/predict.py runs the fine-tuned DistilBERT on the redacted text and returns seven probabilities. On this text you would expect urgency ("before 3 PM today") and secrecy ("don't loop in anyone else") to be high.
6. **Claims.** src/claims/extractor.py finds two claims: c1, affiliation_internal, "this is David from Finance"; c2, payment_request, "process a wire".
7. **Route.** src/router/router.py looks up the table: c1 goes to the header verifier, c2 to the request verifier.
8. **Verify.** src/verifiers/verify.py hands each claim to its verifier. The header verifier (header_verifier.py) fires hv_int_freemail: the email claims to be internal but comes from a free mailbox. Severity high. The request verifier (request_verifier.py) fires rv_freemail: a payment request from a free mailbox outside acmecorp.com; its reason also mentions the Reply-To to protonmail.com. Severity high.
9. **Ledger and score.** src/router/ledger.py collects the rows; src/router/score.py computes the score. The two claims count 60 and then 30 (the second counts half): 90, High risk, before any tactic counts. If urgency or secrecy fired, the multiplier and the tactic points push it to the cap of 100.
10. **Check the report.** Before anything is returned, the pipeline checks its own report: it recomputes the score and the coverage from the ledger and rejects any markup in any text. A mismatch is treated as a server error, never sent as a result. Then the report goes back and the email is forgotten.
11. **The browser draws it.** frontend/src/components/ draws each block: RiskBadge (90, High risk), FindingsTable (the two rows with their reasons), HighlightedBody (the text with the two claims marked), TacticList, ScoreBreakdown (60 + 30), HeaderFindings and CoverageNote

("2 claims found, 2 checked, 2 contradicted"). When the second call returns, the tactic words found by LIME are highlighted too.

The recommended action is fixed text keyed on the strongest finding, roughly: do not reply or pay; confirm with David through a number from your own company directory; report the message to the security team.

The ledger of this email, in short:

Claim	Checked by	Evidence	Verdict
affiliation_internal, "this is David from Finance"	Header verifier	From gmail.com, a free mailbox; authenticated as gmail.com; organisation acmecorp.com	Contradiction, high (hv_int_freemail), 60 points
payment_request, "process a wire"	Request verifier	Payment request from a free mailbox outside acmecorp.com	Contradiction, high (rv_freemail), 30 points (counts half)

Why is there no row for the Reply-To? The Reply-To to protonmail.com is shown in the header findings and mentioned in the payment row's reason, but it is not a separate contradiction. The Reply-To rule runs only for a reply-direction claim ("reply to my private email"), and this body makes none. The same header counts only when a claim makes it meaningful. That is the conditioning idea of Section 3.2 in action. (A slightly longer version of this email in the project's self-test adds "Please reply to my private email only", and then a third, medium row appears.) This was checked by running analyze() on the email.

A variant: the exact-domain spoof. If the attacker had forged From: "David Chen" <d.chen@acmecorp.com> from his own server, the From would look internal. But Acme's DMARC policy would fail the message, and the header verifier would record: claims internal, uses Acme's own domain, fails Acme's authentication, contradiction, high. Either way, the claim is compared with what authentication actually proved.

From the terminal. The same analysis runs without the website: `python -m src.router.pipeline demo/bad_01_wire_request_from_gmail.eml` prints the report for the demo version of this email.

5.2 The hijacked supplier thread, step by step

The demo/ folder holds a made-up supplier conversation (all names, companies and account numbers are invented). Marta Lindqvist of Harbour Freight Supply sends invoice 4471 to Ivan Petrov of Halcyon Freight with the usual bank account (an IBAN starting GB82). Messages 01 to 04 are an ordinary exchange about delivery and a return fee. Then there are two possible fifth messages, both from Marta's real address:

- 05_honest_follow_up_marta.eml: a normal reply.
- 05_HIJACK_new_bank_account_marta.eml: the hijack.

The hijacked message reads, in part:

```
From: Marta Lindqvist <marta.lindqvist@harbourfreightsupply.com>
Authentication-Results: mx.halcyonfreight.com; spf=pass ... dkim=pass ...
    dmarc=pass header.from=harbourfreightsupply.com
In-Reply-To: <t1.4@mail.halcyonfreight.com>
X-Mailer: Roundcube Webmail 1.6.5

Quick update: our bank has changed this week and the old account is
closed. Please use our new account for invoice 4471: IBAN DE89 3704 ...
Pay today, it is urgent, or the delivery on Thursday may be held. This is
confidential until our audit is finished, so please do not mention it to
anyone at our company and reply only to me.
```

Why single-message checks see nothing. The address is Marta's real one. SPF, DKIM and DMARC all pass for her own company's domain. There is no free mailbox, no lookalike, no Reply-To elsewhere. The In-Reply-To points to the real previous message, and the quoted text matches what Ivan really wrote. On its own, this is a well-authenticated email from a known supplier. The header and request verifiers find nothing to contradict.

What the thread verifier sees when you load all five files (the order does not matter; the newest message by its Date header is judged against the others):

- **Request drift:** a bank account that never appeared in the thread (DE89...) replaces the one that did (GB82...), together with a payment-change claim.
- **Tactic onset:** urgency ("pay today, it is urgent") and secrecy ("do not mention it to anyone... reply only to me") appear for the first time in a thread that was calm for four messages.
- **Sending-path drift:** Marta's earlier messages were written with a different mail program; this one names Roundcube Webmail. On its own this is weak (people change programs), but together with the rest it fits the pattern.
- **Thread integrity** stays consistent: the attacker is inside the real account, so the IDs and quotes are real.

The scan marks **message 5 as the point where the thread flipped**, and the band is Suspicious to High risk depending on what the tactic model gives. Load messages 01 to 04 with the honest fifth message instead, and the result is Low risk: no false alarm.

From the terminal: `python -m src.router.pipeline --thread demo/thread_supplier_payment/0[1-4]_*.eml demo/thread_supplier_payment/05_HIJACK_new_bank_account_marta.eml`

> **This is the case no single-message check can see.** The account is genuine, so every authentication check passes, and only the conversation history gives it away. It is exactly what N2 is for.

5.3 The other demo emails

The demo/ folder also has ten single emails, six bad and four good. The most instructive pair is bad_01 and good_01: almost the same words (a wire request with urgency and secrecy), but good_01 comes from the finance director's own company address with passing authentication. bad_01 lands Suspicious to High risk, good_01 Low risk. The score difference comes from the headers, not the words. The others show a CEO gift-card request from a lookalike domain with a digit 1 in place of the letter i, a forged company address that fails SPF and DMARC, a payroll-details request from outlook.com with a Reply-To to yet another domain, a supplier invoice with a new bank and a Reply-To to Gmail, a "PayPal Service" password request from paypa1.com, an

ordinary supplier question, a mailing-list digest and a colleague who says "it is a bit urgent" about routine forms.

6. What is new: N1, N2, N3 and the architecture

A student project is judged partly on novelty: what it adds that did not exist before. The novelty was fixed on 21 September 2026 after the first review, and it has four parts. For each: what it is, why it matters, how it is tested, and what it does **not** claim.

ID	Name	One line
N1	Payload-free evaluation protocol	Hide links, addresses, domains and file names, and measure how much of a detector's accuracy was link-reading
N2	Thread consistency verification	Check a reply against its own conversation and point to the message where the thread flipped
N3 (core)	Claim-evidence contradiction engine	Check each claim in the body against what the headers can prove, with the check conditioned on the claim
Architecture	Claim-routed verification pipeline	Route each typed claim to the checker that can test it; the output is a ledger of claim, evidence and verdict

> **One line to remember.** N3 catches the impersonator. N2 catches the attacker who does not need to impersonate, because he is already inside the account.

6.1 N1: the payload-free evaluation protocol

What. Before any model trains or predicts, the body is redacted: links become [URL], addresses [EMAIL], file names [FILE], domains [DOMAIN] (Section 4.3).

Why it matters. Published phishing detectors often report 97% to 99% accuracy. But in phishing collections a suspicious link predicts the label, so a model can score high by reading links instead of understanding manipulation. Real pretexting has no link, so that shortcut does not exist in the attack that matters. N1 measures how big the shortcut is.

How it is tested. Two DistilBERT models learn the same task (attack against not-attack). Model A trains on raw bodies, the way earlier work does; model B on redacted bodies. Both are tested on the raw view, the redacted view and the emails that never had a link. If A loses accuracy when links are hidden and B does not, A was partly reading links. The same is repeated with the cheaper TF-IDF models as a check.

What it does not claim. It does not claim that redaction makes a better detector of real payload-free fraud; there is no public collection of such fraud to show that. It claims a measurement: how much of a raw-trained detector's score depends on links. The result (Part 12): yes, measurably, in all three model families.

6.2 N2: thread consistency verification

What. A reply is compared with the earlier messages of its own thread on four signals: tactic onset, request drift, sending-path drift and thread integrity (Section 4.7). The output is the message where the thread flipped and the signals that fired.

Why it matters. An attacker inside a real account passes every header check, so a single-message detector cannot see them. Academic detectors classify single emails. The closest academic work on compromised accounts (Ho et al., 2019) targets emails with a malicious link

and uses data from a whole organisation. Commercial tools that analyse threads are closed and rely on login data. We did not find an open, explainable detector that checks a message against its own thread using the thread's headers.

How it is tested. No public set of real hijacked threads exists, so the project built a benchmark: real threads from Enron and the Apache lists, with a synthetic message put in place of a real reply in three variants (A, account takeover with drifting content; B, a lookalike domain swap; C, a forged thread), and two kinds of honest negative (the real next reply, and a synthetic benign continuation).

What it does not claim. It does not claim to catch real attackers at a measured rate: the injected messages are made up, written from the same fields the signals read. It does not catch a careful hijacker who has read the mailbox and copies the real IDs, the sending details and the calm tone. That case is outside what any rule here can catch, and the project says so.

6.3 N3: the claim-evidence contradiction engine (the core)

What. Affiliation, identity and authority claims in the body are checked against the header evidence that could confirm or contradict them (Section 4.7, header verifier). The check is conditioned on the claim: the same headers mean different things under different claims (Section 3.2).

Why it matters. Other designs blend evidence without knowing which claim it is about. BEC-Guard (Cidon et al., 2019), a well-known commercial detector, first decides from an organisation's header history whether a sender impersonates an employee, and only then classifies the body, after removing names and signatures from it. Because the statements that say who is writing are removed, no step compares what the email claims with what its headers show. A "parallel fusion" design simply averages a text score and a header score. Neither can say "the body claims X and the headers disprove X".

How it is tested. The frozen claim-routed score is compared with three learned systems that use the same information in simpler ways: text only, headers only, and parallel fusion of the two. Real attacks from three collections, plus a synthetic arm of colleague-impersonation emails whose headers were written by rule, reported apart.

What it does not claim, and what happened. It was expected to detect more attacks than the simpler systems. **It did not.** On pooled test data it detects fewer (Part 12). What the design does give is a stated reason for every alarm, and on the synthetic colleague-impersonation pairs it flagged attacks without flagging their honest twins, where the headers-only system flagged both. The project, and the paper, report this as it is.

6.4 The architecture contribution: claim-routed verification

What. Most detectors work as email to features to classifier to score, with an explanation added afterwards. PretextGuard works like a fact-checking pipeline: detect the claims, fetch the evidence for each, give a verdict with its justification. Automated fact-checking research divides its task the same way (claim detection, evidence retrieval, claim verification). Here the evidence is protocol metadata and conversation history instead of documents.

Three properties follow:

1. **Claim-first evidence.** The body decides which evidence gets checked, claim by claim.
2. **Explanation by construction.** The output is the reason, written by the rule that found it.
3. **Extensible.** A new attack pattern means adding one rule; the model does not need retraining.

Design style	Example	Weakness
End-to-end classifier	Tooher and Lallie's two-stage framework (2025)	Score first, explanation added afterwards
Gated pipeline	BEC-Guard (2019)	Names and signatures removed, so a claim cannot meet its evidence
Parallel fusion	The N3 comparator	Two scores averaged; neither knows which claim the other is about
Language-model judgement	ChatSpamDetector (Koide et al., 2024)	One expensive model call per email; emails without links were excluded
Claim annotation only	Paul et al. (2024)	Labels the warning signs, leaves the comparison with metadata to "a downstream verification algorithm"
Claim-routed verification	PretextGuard	This project

How it is tested. The same signals are wired two ways: the routed score, and a flat classifier that gets all the signals in one list without the link between a claim and its evidence. The flat classifier detects more; the routed design is the one that can name a claim and a rule behind every alarm (Part 12).

6.5 Plain parts, not claimed as new

These are useful but not novel, and the project never presents them as novelty: the 0 to 100 risk score, the DistilBERT tactic classifier, the seventh tactic (isolation or secrecy), LIME highlights and the keyword baseline.

6.6 Ideas that were dropped, and why

Knowing what was rejected explains why the project looks the way it does.

Idea	What happened	Why
Version 1: DistilBERT trained on emails written by a language model and labelled with Cialdini's principles	Scrapped in July 2026	Almost the same as a published 2025 paper (Tooher and Lallie)
Transferring persuasion labels from a news data set (SemEval-2023) as a novelty	Struck by the faculty after the first review	Kept only as optional pretraining, which was finally skipped
A tactic-stacking severity model as a novelty	Struck by the faculty	Kept only as the plain 0 to 100 score
The technology choice (FastAPI, React, no database) as a novelty	Never valid	The faculty's rule excludes stack choices
Tactic co-occurrence analysis as a novelty	Not valid	Already done in published work

7. The data

7.1 Why so many sources

The project needs five things, and no single public collection has them all:

1. Examples of ordinary email and of attacks.
2. Real headers, including modern authentication verdicts.
3. Tactic and claim labels (no email collection has them).
4. Real conversations, for the thread signals that read content.
5. Real conversations with full headers, for the thread signals that read headers.

So each source fills a gap. The principle was **real data first; synthetic data only to fill gaps that are measured and written down**.

7.2 The nine sources

All emails go into one table (data/processed/staged.parquet) with a source, a **category** (ham, spam, phishing or fraud) and an attack flag, which is true only for phishing and fraud. Spam is not counted as an attack: it is a nuisance, not a pretexting attack, and counting it would pad the attack side with easy cases.

Source	What it is	Kept after removing copies	Category	Headers
Kaggle CEAS-08	Email from a 2008 spam-filtering challenge, from a merged Kaggle collection	38,077	Ham and spam	Only sender, receiver, date, subject
Kaggle Enron	Company email from Enron (2001)	29,119	Ham and spam	Subject only
Kaggle Ling	An older mailing-list collection	2,850	Ham and spam	Subject only
Kaggle Nigerian Fraud	Advance-fee fraud emails	3,227	Fraud	Only sender, receiver, date, subject
SpamAssassin public corpus	A classic spam-filter test set (2002 to 2005), raw files	5,775	Ham and spam	Full raw headers, but from before DMARC existed
Nazario phishing corpus	A researcher's long-running phishing collection, up to 2025, raw files	9,595	Phishing	Full raw headers; about a fifth carry authentication results
phishing_pot	Real phishing caught by honeypot mailboxes, on GitHub	7,491	Phishing	Full raw headers; 99.5% carry authentication results
Apache Tomcat users list	A real software mailing list, Oct 2024 to Sep 2026	2,135	Ham	Full headers with DKIM verdicts and reply IDs
Apache Kafka users list	Another real software mailing list, same period	1,055	Ham	Full headers with DKIM verdicts and reply IDs

Total: 99,324 unique emails, of which 42,354 ham, 36,657 spam, 17,086 phishing and 3,227 fraud (20,313 attacks).

Two more sources were used but are not in this table: the **raw Enron collection** (517,401 messages) for rebuilding real threads, and the **synthetic emails** described below. A news data set with human persuasion labels (SemEval-2023 Task 3) was mapped to our tactics but finally not used.

Licences. phishing_pot is licensed for non-commercial use with attribution, and it is never redistributed: the annotation batches with full email text are not committed to GitHub.

7.3 Cleaning up the collections (Phase 1)

- **Copies removed.** The Kaggle merge repeats emails that the raw downloads also contain. Every body got a fingerprint (a SHA-256 hash of its letters and digits, so copies that differ only in spacing match) and one copy was kept. 5,484 copies were removed. The same email in train and test would inflate every score.
- **Two Kaggle files left out.** Kaggle's Nazario and SpamAssassin files turned out to be reprocessed copies of the raw corpora (line breaks collapsed, some messages run together), so the fingerprint missed them. A check by sender, date and subject matched 98% and 96% of them. Keeping them would have leaked copies of training emails into the test set, so the raw originals were used instead.
- **Empty emails dropped.** 208 emails with no readable text were dropped. Attachments are never decoded.

7.4 Which sources have which headers

Phase 1 measured, for every source, the share of emails carrying each header (results/header_coverage.csv). The key facts:

- Authentication-Results: 99.5% of phishing_pot, about 100% of the Apache lists, 20.4% of raw Nazario, 0% of SpamAssassin and raw Enron. The Kaggle rows carry no original headers at all.
- Among the legitimate sources, **only Apache has authentication verdicts, and only DKIM**. No legitimate source has SPF or DMARC verdicts.

This imbalance has a big consequence. A model that learned from authentication verdicts would learn "has verdicts means attack", which is a fact about the collections, not about attacks. So authentication evidence is **never** a learned feature in PretextGuard; it is used only inside claim-conditioned rules, and every result involving it is reported per source.

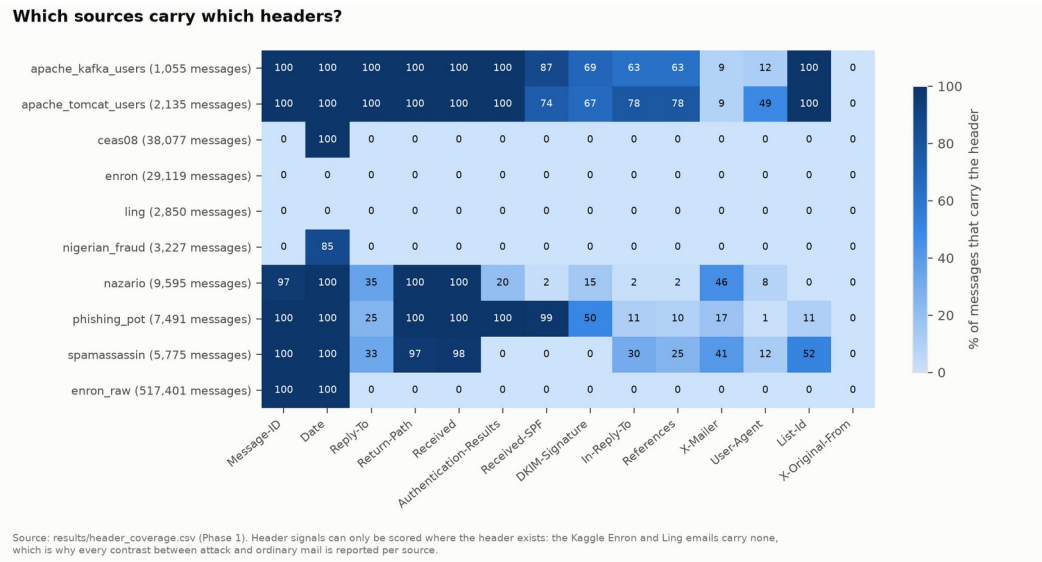


Figure 5. Header coverage per source (docs/figures/results/fig_header_coverage.png)

7.5 The split

The table was split 70% train, 15% validation and 15% test, separately for each source and category (so every source appears in every split). Emails whose subjects match after removing "Re:" and "Fwd:" (a conversation, or one spam campaign) were kept in the same split, so near-copies never sit on both sides. The order came from a fixed hash (seed 42), so the split is identical on every machine. Result: **69,542 train, 14,879 validation and 14,903 test** emails. The test split was not used until Phase 13.

7.6 The labels for tactics and claims (Phase 5)

No email collection has tactic or claim labels, so the project made them.

- **The sample.** 700 real emails: 450 attacks (150 each from phishing_pot, Nazario and Nigerian Fraud), 150 ham and 100 spam, a fixed number per source and category so that small sources are not drowned out, drawn from all three splits (60/20/20) so that validation and test have labels too. Ham and spam are labelled as well, because a tactic model that never sees ordinary urgent mail would fire on it.
- **The labellers.** Two Gemini models (gemini-3.1-flash-lite and gemini-3.5-flash-lite) labelled every email separately through a free API at temperature 0 (the setting that makes a model's answer as repeatable as possible), in 35 batches of 20, seeing only the redacted body. A third model (gemini-flash-lite-latest) decided where they disagreed. Every reply had to be a JSON list with a fixed shape, and every quoted claim had to appear in the email; a bad reply was asked again once and then dropped.
- **Agreement.** Mean Cohen's kappa between the two labellers was 0.501 over the tactics and 0.458 over the claim types: moderate. Urgency (0.694), scarcity (0.790) and secrecy (0.692) agreed well; social proof (0.262) and reciprocity (0.266) poorly.
- **Result.** 690 of the 700 emails were labelled: 413 in train, 137 in validation and 140 in test.
- **The limits.** All three labellers are models of one family from one provider, so their mistakes are partly shared; the tie-breaker's name is a moving alias, so its exact version is unknown; and no person checked the labels. You chose not to hand-label (a decision recorded in July 2026), so every tactic and claim score in this project means "agreement with these model labels", not "correct".

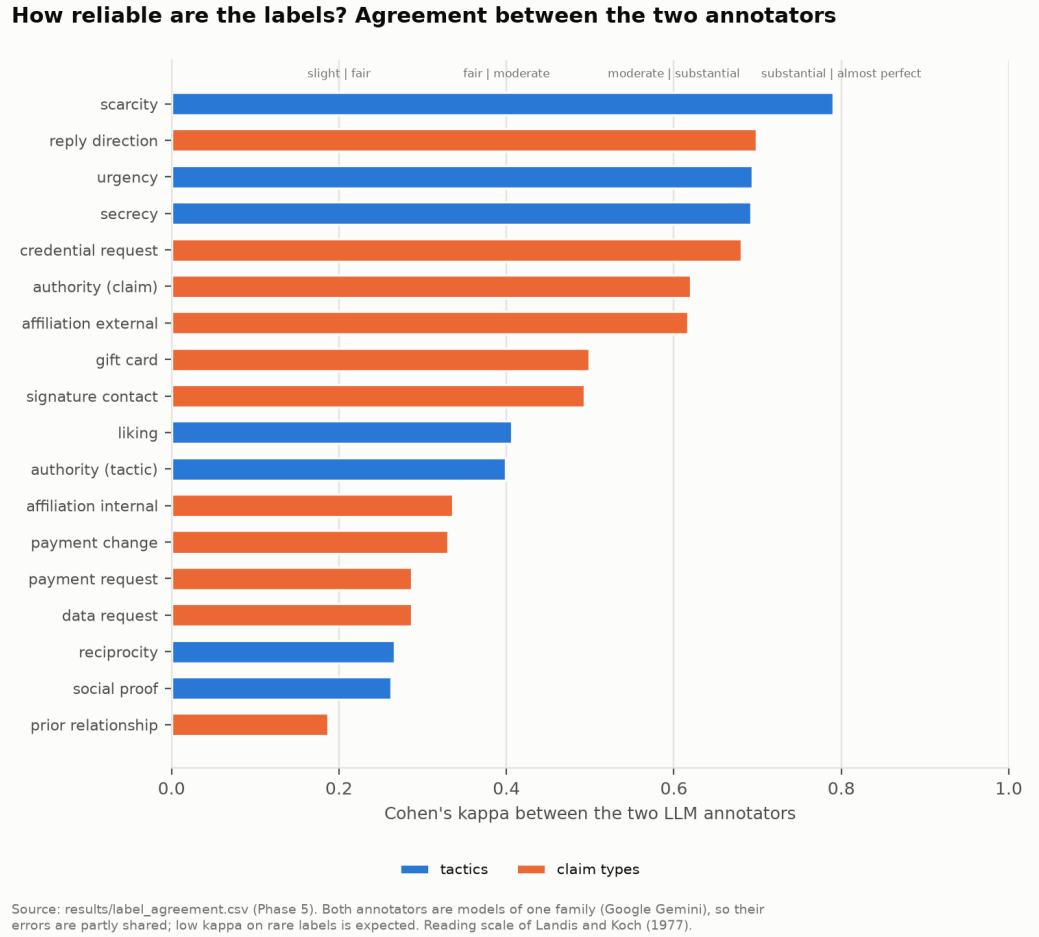


Figure 6. Agreement between the two labelling models (docs/figures/results/fig_annotator_agreement.png)

7.7 The synthetic emails

Three tactics (reciprocity, social proof, liking) are almost absent from real emails: the test split has 0, 0 and 4 real examples of them. To give the model something to learn from, a language model wrote **222 synthetic pairs** (444 emails): an attack and a **benign twin** from the same template, over 12 situations, with each attack's tactics fixed by the plan and confirmed by a quoted phrase. Every synthetic email was checked: no link, 20 to 260 words, every required claim quoted from the body, and the twin must not trigger the keyword baseline. Pairs were split by pair, so a pair never straddles train and test.

Synthetic results are always reported apart from real ones, because they carry one model's writing style. The benign twin exists for the same reason: if attacks are written by a language model and legitimate emails by people in 2001, a model can learn "language model versus 2001" instead of manipulation. Giving every synthetic attack a synthetic twin from the same generator controls for that.

7.8 The threads (Phase 9)

- **Apache:** threads rebuilt exactly from Message-ID, In-Reply-To and References.
- **Enron:** threads guessed the way old mail programs did it: the subject without "Re:", cut into runs where a gap of more than 14 days starts a new run, joined when messages share a participant, with copies across folders removed. 1,200 Enron threads were kept.
- **Rules for a thread:** 3 to 50 messages and at least two senders. In total 1,591 threads (391 Apache, 1,200 Enron), split by thread so no thread appears in two splits.

Raw Enron has no reply IDs and no sending-path headers, so Enron threads test the content signals (tactic onset, request drift) and the quote check; Apache threads test all four signals.

The **hijack benchmark** then built five cases per base thread: the three attack variants (A takeover, B lookalike swap, C forged thread; B only on Apache, because Enron has no sending-path headers) and two negatives (the real next reply, and a synthetic benign continuation). The injected texts were written by the same family of language models and checked by script.

7.9 The N1 sample

Training two DistilBERT models on all 69,542 train emails would take about three hours on Colab's free GPU, so the N1 models were trained on a sample: at most 800 emails per source and category, **9,856 emails, of which 2,400 are attacks**, with two seeds each. The TF-IDF models were trained both on the same sample and on all train emails, which shows what the sample costs. The N1 labels are **corpus labels**: an email counts as an attack because its collection says so (phishing or fraud), not because any model labelled it.

8. A tour of the project folder

The repository (github.com/Nagasai-Datta/PretextGuard, and your copy at ~/Desktop/pretextguard) is organised like this. You do not need to know every file; the ones listed are the ones that matter.

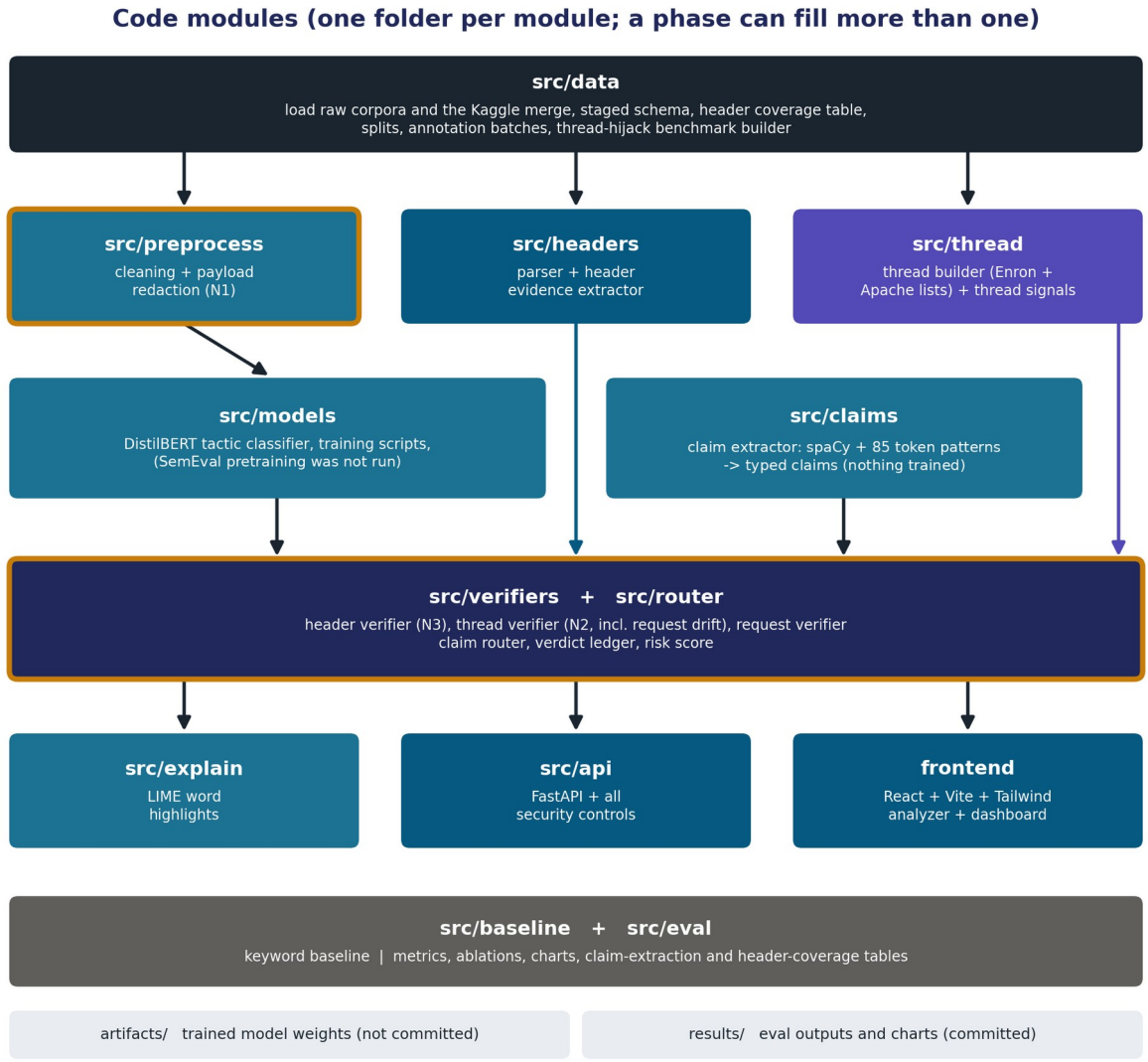


Figure 7. The code modules and how they feed each other (docs/figures/figure4_module_map.png)

8.1 The Python side (src/)

Folder	What lives there	Files worth knowing
src/data/	Getting, staging and splitting the data; labels; synthetic emails; the hijack benchmark	stage.py (the big table), split.py (the 70/15/15 split), coverage.py (header coverage), annotate.py and llm_api.py (labelling through the free API), agreement.py (kappa), labels.py (final labels), synthetic.py, hijack_benchmark.py
src/preprocess/	Cleaning and redaction (N1)	clean.py (HTML to text, footers, quotes, signature), redact.py ([URL] [EMAIL] [FILE] [DOMAIN])
src/headers/	Parsing headers and building the evidence	parser.py, domains.py (registered domains, freemail list, lookalike score), evidence.py (trusted authentication verdicts and the rest)

Folder	What lives there	Files worth knowing
src/baseline/	The keyword baseline	lexicon.py (the word lists), keywords.py (the scorer)
src/models/	The DistilBERT tactic classifier	dataset.py, train.py (run on Colab), predict.py (used at run time), validate.py
src/claims/	The claim extractor	patterns.py (the 85 token patterns), extractor.py
src/verifiers/	The three verifiers and the ledger row	rows.py (the row and the routing table), header_verifier.py (N3), request_verifier.py, thread_verifier.py (N2), brands.py (51 brands' domains), bank.py (IBAN checks), verify.py
src/thread/	Rebuilding threads and the thread signals	builder.py, signals.py (the four measurements)
src/router/	Routing, ledger, score and the whole pipeline	router.py, ledger.py, score.py, reliability.json (the rules that count half or zero), pipeline.py (analyze(), the heart of the system)
src/explain/	LIME	lime_explain.py, check.py (the deletion test)
src/api/	The FastAPI server	main.py (the routes), security.py (key, rate limits, audit log), schemas.py, settings.py
src/eval/	Everything for Phase 13	freeze.py (the freeze record), stats.py (bootstrap, intervals, AUC), one script per experiment (tactic_test.py, claim_extraction.py, score_test.py, ablation_n1.py, ablation_n2.py, ablation_n3.py, ablation_arch.py, style_confound.py, paraphrase.py), charts.py, rerun_check.py, selftest.py

Many folders also have a selftest.py: a quick script that feeds hand-made inputs to the code and checks the outputs, with no real data needed.

8.2 The rest

Folder or file	What it is
frontend/	The website (React, Vite, Tailwind): src/pages/ (AnalyzerPage, DashboardPage), src/components/ (one component per block of a result), src/lib/ (the two-call flow, text offsets, size limits), src/api.js (the only network code), vite.config.js (the proxy and the security headers), scripts/ (static checks, mutation check, browser check)
results/	Every result file, about 40 CSV and JSON files for Phase 13 alone, committed to GitHub. Every number in the paper comes from here
docs/	master_document.md (the source of truth), PretextGuard_Context.md (the handover file for a new chat), the Word copy of the master document, figures/ (five design figures), figures/results/ (13 result charts), paper/ (the paper and its sources), and this explainer
notebooks/	The two Colab notebooks: phase6_tactic_classifier.ipynb and phase13_n1_models.ipynb

Folder or file	What it is
demo/	Ten made-up emails and the hijacked supplier thread, to try by hand (Part 5)
data/	README.md and the small files that can be shared: the label sample, raw labeller replies, synthetic emails, thread benchmark plans. The big files (raw downloads, about 3 GB, and the processed tables) stay on the Mac
artifacts/	The trained model (about 270 MB). Only its README is in GitHub; the weights stay on the Mac
tests/	One environment check (16 checks that the setup is right)
README.md, CLAUDE.md, requirements.txt	The overview; a pointer that tells Claude Code to read the context file; the exact Python library versions

Two things are deliberately **not** in the repository: **.env**, which holds the API key and the Gemini key, and the **venv/** folder, the Python environment (the Python version of node_modules), which can be recreated from requirements.txt.

8.3 The data tables

Each phase writes its own file and never changes an earlier one, so a mistake in one phase cannot damage another:

File	Written in	Holds
data/processed/staged.parquet	Phase 1	The 99,324 emails with source, category, raw headers, raw body and split
data/processed/cleaned.parquet	Phase 2	The same plus the clean body, the redacted body, the signature and has_url
data/processed/headers.parquet	Phase 3	One row of header evidence per email
data/processed/threads.parquet	Phase 9	The rebuilt threads
data/labelled/labels.csv	Phase 5	The final tactic and claim labels of the 690 emails

Parquet is a compressed table format that keeps column types; think of it as a CSV file that is smaller, faster and knows which columns are numbers.

9. The technology, one line each

Technology	What it does here	Why this one
Python 3.12 in a venv	The language of everything except the website	The model lives in Python; the venv keeps the project's libraries separate, like node_modules
pandas and Parquet (pyarrow)	Tables in memory and on disk	The standard tools for tabular data
Python's email package	Parsing headers and bodies	Built in, and forgiving with broken spam headers
BeautifulSoup	HTML to text	Copes well with messy HTML
tldextract	Splitting domains with the public suffix list	Works offline with its built-in list, so it never downloads anything at run time
RapidFuzz	Lookalike scores between domains	Fast string similarity
spaCy (small English model)	Tokens and names of people and organisations	Fast; token patterns avoid the ReDoS risk of regular expressions
PyTorch and Hugging Face Transformers	Fine-tuning and running DistilBERT	The standard pair for transformer models
Google Colab	Training on a free GPU	No local GPU needed; training ran once per model
scikit-learn	The TF-IDF and logistic regression comparators, and cross-checks of hand-written metrics	The standard library for classic machine learning
Gemini models through a free API	Labelling emails and writing synthetic emails, offline only	Free; never used at run time
FastAPI, Pydantic, slowapi, uvicorn	The API server: routes, request schemas, rate limits, the server itself	FastAPI is Python's closest equivalent to Express; Pydantic is like Joi or Zod
React, Vite, Tailwind, Recharts	The website, its build tool and dev server, its styling, its charts	Reuses React you already know; plain JavaScript; no router, state library or UI kit, so every line can be explained
matplotlib	The 13 result charts for the paper	The standard Python plotting library
pip-audit and npm audit	Checking libraries for known security problems	Dependency hygiene
Git and GitHub	Every version of the project, public	The commit history shows the work as it happened

No database, on purpose. The tool handles sensitive company email; storing it would create the very breach risk the tool exists to reduce. Each email lives in memory for one request.

No language model at run time, on purpose. Cost and delay at the scale of a mail gateway, no way to run it inside a company's own network, answers that vary from run to run, and no inspectable reasoning. A language model was considered only as a comparison.

Rejected: the MERN stack. Express would only have passed requests through to the Python model, and MongoDB would have stored the email content the design promises not to keep.

Versions are pinned exactly (requirements.txt for Python, package-lock.json for the website), so the project installs the same way on any machine.

10. Security of the system

Security was treated as a first-class part of the project (it was worth 15 marks in the course rubric). The basic stance: **everything in an email is written by the attacker**, including most of its headers, so every piece of it is handled as hostile input. Here are the controls in plain words, grouped by where they act.

10.1 Handling attacker-written email

- **No regular expressions over email text where they could blow up.** A carelessly written pattern can take hours on a crafted input (ReDoS). The cleaning code has bounded patterns only and caps bodies at 200,000 characters. Testing 31 crafted inputs found two real bugs (a footer pattern that ran for hours on 200,000 dashes; 10 seconds on 20,000 nested HTML tags); after the fixes the slowest input takes under two seconds. The keyword baseline, the claim extractor, the verifiers and the thread code avoid regular expressions over email text altogether.
- **Headers are capped and parsed one field at a time.** The header block is cut at 64 KB, each field at 2,000 characters, with at most 50 Received lines and 10 Authentication-Results headers kept.
- **Only trusted authentication results are read** (Section 2.4). A fake dmarc=pass written by the sender is ignored.
- **Attachments are never opened.** Nested email parts are limited to 20 levels: a test with 3,000 nested parts crashed Python's own parser, so deeper mail is now read as plain text with a note.
- **Threads are bounded.** At most 50 messages, IDs only looked up and never followed, so a looping reply chain cannot trap the code.

10.2 The API

Control	What it does
API key	Every POST route needs the key in an X-API-Key header. It is compared as a hash in constant time (so timing cannot leak it), never read from the URL. The server refuses to start with a weak key. Every kind of failure gets the same answer
Rate limits	Per client address: 120 requests a minute in total, 30 for the analyse routes, 6 for /explain. Counted before the key is checked, so guessing keys uses up the allowance
Size limits	At most 4,000,000 bytes per request, counted as the bytes arrive (a sender can lie about the size), 300,000 per email, 50 messages and 1,500,000 bytes per thread
Strict input	JSON only (a form posted from another website cannot reach the routes); schemas refuse unknown fields and wrong types
One analysis at a time	A lock, with a "busy" answer after 2 seconds instead of an endless queue
Safe output	Every string from an email is made display-safe before it enters a report (angle brackets replaced, invisible characters removed); reasons come from fixed templates; the report checks itself before it is sent
Response headers	No caching, no content sniffing, no framing, a strict Content-Security-Policy, a request ID on every answer

Control	What it does
Audit log	One line per event from a fixed list of fields (request ID, time, score, verdict and so on). It cannot hold email content: a test sends a marker string in every part of a request and checks it appears in no log line
No storage	The email is held in memory for the request only, never logged or written to disk

CORS is open to any origin, deliberately. CORS_ALLOW_ANY_ORIGIN = True in src/api/main.py is a deliberate setting: no credentials are allowed, and the key is still required on the POST routes. It is a plain fact of the design, not an oversight.

10.3 The website

- **No script injection (XSS).** Email text is attacker-controlled, so every string is drawn as plain React text; nothing in the code inserts raw HTML. A test drives an email with script payloads in every part (display name, subject, body, signature) through the built app in a real browser and checks that no script runs.
- **The key never reaches the browser.** The small server that serves the page adds the key when it forwards a request to the API. A check searches the built files for the key.
- **A strict Content-Security-Policy** on the served page: it may load only its own scripts, styles and images, and connect only to its own server. No CDN, no web fonts, nothing stored in the browser.

10.4 Build time

- **Downloads unpacked safely:** archive paths checked so a file cannot escape its folder, a 5 GB limit, file types checked by their first bytes, raw data made read-only.
- **Model files are safe to load:** weights are stored as safetensors (plain numbers), not the older pickle format that can run code when loaded; loading never downloads anything.
- **Prompt injection during labelling:** an attacker's email could contain "ignore your instructions". Each email was wrapped as data in a block it cannot close, the prompt said never to follow text inside it, and only replies with the exact expected shape were accepted.
- **Redaction before data leaves the machine:** emails sent to the free labelling API were redacted first (no real links or addresses).
- **Secrets** live in .env, which Git ignores; .env.example lists the names only.

10.5 Do the checks themselves work?

A test suite that never fails proves nothing. So the project also ran **mutation checks**: break one control on purpose and see whether a test notices. The API's tests noticed 37 of 37 broken controls; the website's checks noticed 26 of 26. The API also passed 88 self-test checks and 19 checks in a real session with the trained model, and the built website passed 93 checks in a real browser.

Known limits (from the master document). The system is built as a single-user demo on one machine, and a real deployment would need more:

- The API listens only on the machine itself and speaks plain HTTP. An exposed deployment needs a reverse proxy in front that adds HTTPS and request timeouts (25 connections that never finish sending their headers made the server answer "busy" until they closed).
- The rate limits and the lock live in memory, so they hold for one server process; behind the website's proxy all users share one allowance.
- The Content-Security-Policy is sent by the preview server, not by the development server.
- Only a Chromium-based browser was tested; Safari, Firefox and screen readers were not.

11. How it was built

11.1 How the work was done

The project was built in phases, each in its own chat with Claude. The pattern was the same every time:

1. Claude explained the background of the phase and proposed a plan; nothing was built before you said go.
2. Claude delivered every file, with a function-by-function explanation and the exact commands to run.
3. You ran everything on your Mac in VS Code (training on Colab), reported the output or the error, and fixed until it worked.
4. You committed and pushed. Claude never ran Git; it read your pushed results from GitHub.
5. At the end of the phase, the master document recorded what was built, the results and every decision, with a new version number.

A new chat has no memory of older chats, so every chat started from two files: docs/PretextGuard_Context.md (the handover: how to work, the environment, the current state) and docs/master_document.md (the single source of truth). If anything disagreed with the master document, the master document won.

11.2 The phases

Phase	Finished	What was built	Why, and a key outcome
Review-I	From 29 Jul	First review with the faculty (a slide deck, about 10 minutes)	The faculty struck two earlier novelty ideas and asked for one more core claim; N1 and N3 survived, N2 and the architecture were added, and the novelty was locked on 21 September
0	Sep 2026	The environment: Python 3.12 venv, VS Code, Git, folders, pinned requirements, .gitignore, .env.example, an environment test, pip-audit	A clean, repeatable setup before any data
1	6 Oct	Getting the data: downloads, safe unpacking, the Apache archives, one reader per format, copies removed, the header coverage table, the split	One table of 99,324 unique emails from nine sources; 5,484 copies removed
2	6 Oct	Cleaning and redaction (N1)	The clean and redacted bodies; 4,580 attacks are naturally link-free (696 in test), enough for N1's link-free view
3	7 Oct	The header parser and evidence	Trusted authentication verdicts, freemail, lookalike scores; the fallback raised readable phishing_pot From lines from 72.0% to 92.1%; found that no legitimate source has SPF or DMARC verdicts

Phase	Finished	What was built	Why, and a key outcome
4	7 Oct	The keyword baseline	546 phrases for the seven tactics, written before any label existed, so the comparison with DistilBERT is fair; 20 of 20 sanity checks
5	8 Oct	Labels and synthetic emails	690 labelled real emails (mean kappa 0.501 on tactics, 0.458 on claims); 222 synthetic pairs
6	8 Oct	The DistilBERT tactic classifier, trained on Colab in 22.2 minutes	Validation macro-F1 0.613 against 0.495 for the tuned keyword baseline (137 emails; slightly optimistic, because validation chose the settings)
7	8 Oct	The claim extractor	85 token patterns in four versions, written from train emails only; version 0.4 frozen before validation was scored once
8	9 Oct	The header verifier (N3) and the request verifier	Rules frozen at version 0.3. On train, emails with a contradicted claim: ham 3.5%, spam 1.7%, phishing 18.4%, fraud 40.9%
9	9 Oct	Thread rebuilding, the hijack benchmark and the thread verifier (N2)	44 rules frozen at version 0.2; on real validation threads, medium or high false alarms on 1.1% of Apache and 4.8% of Enron messages that had earlier messages
10	9 Oct	The router, the ledger, the risk score and LIME	Score version 0.2 frozen; the false-alarm budget held in 5 of 6 validation groups (the miss, Enron thread messages, was reported, not hidden)
11	10 Oct	The FastAPI server with all security controls	88 self-test checks; 37 of 37 deliberately broken controls noticed; 19 checks in a real session
12	10 Oct	The React website: analyzer and dashboard	96 static checks; 26 of 26 deliberate breakages noticed; 93 checks in a real browser
13	10 Oct	The evaluation on the test split: every experiment, 13 charts	Part 12
14	Cancelled	Report, viva preparation, review deck	Your faculty evaluates the project from the paper (decided 15 October)

11.3 A rule that ran through every phase: frozen before read

Every component that makes a decision (the redaction, the keyword lists, the claim patterns, the verifier rules, the thread rules, the score numbers, the model and its thresholds) followed the same discipline:

- It was developed by reading the **train** split only. The development scripts have a --train-only mode that never loads a validation email.
- It was **frozen** (given a version number and committed) before the **validation** split was read, once.

- Nothing was changed after the **test** split was read in Phase 13.

When something was changed after validation was read, the master document says so. The clearest case is in Phase 10: the first validation run counted spam as "ordinary mail", which made the core fraud rule count only half (spam asks for money from free mailboxes too). The definition was corrected to legitimate mail only and the run repeated, and the master document marks the validation figures as "a check, not an independent test" for that reason. The clean measurement is the test split.

11.4 After Phase 13: the paper, and the two bugs

Between 12 and 15 October the work moved to the conference paper. This is what ran, and why.

What	What it did	Why
Literature check (12 Oct)	Every reference in the master document was looked up on a free page and at least its abstract read; one could not be found for free and was dropped; 19 new ones were added; the close neighbours were read in full. The result is the source table and the literature notes in docs/paper/	A paper may cite only what was actually read, for what it actually says
Master document 3.15.2 and 3.15.3	Corrected statistics and the descriptions of two close works (BEC-Guard and Pan et al.)	The reading showed the earlier descriptions were wrong (for example, BEC-Guard is a gated pipeline, not two scores added)
Paper plan and drafts (13 to 15 Oct)	Draft 1 in the IEEE template, then drafts 2 to 4	Part 13
Two reporting bugs found (15 Oct)	While planning the paper's per-source table, two bugs in evaluation scripts turned up (below)	The paper wanted to report results per source
Fix package (commit 258cb0c)	Fixed four evaluation scripts and added checks for both bugs to the scripts and to the self-test (110 of 110 checks passed)	A bug in an evaluation script may be fixed and the script rerun, with the reason logged; a frozen component may never be changed
Logged reruns (commit 18cec7f)	Reran the four scripts once each on the test split, with the --rerun flag and the reason written into the test log	To produce correct per-source tables
rerun_check.py	Compared every output file with the copies taken before the reruns: every pooled number, every difference and every other table identical (25 checks, 0 failures); the freeze record still matched (63 items, nothing differs)	Proof that the fix changed only the per-source rows it was meant to change
Master document 3.15.4 to 3.15.6	Recorded the reruns, the corrected per-source table, the paper drafts, the faculty's decision and the close of the project	The master document is the source of truth

The two bugs, in plain words.

1. Four sources hold both ham and spam (CEAS-08, Kaggle Enron, Ling, SpamAssassin). Three scripts (the N1, N3 and architecture ablations) labelled each per-source row with the category of the source's first email, so each of those four sources got **one row covering ham and spam together**, labelled as one of them. Now there is one row per source and category.
2. The style-confound script, when comparing two sources of the same kind (two ham sources, say), read **every** email of each source, so a "ham pair" also read the spam of those sources, and six "spam pairs" repeated six ham pairs. Now each pair reads only its own kind.

Neither bug touched a frozen component, and no pooled number changed. Only the per-source table of N3 and the style-confound line changed. The paper reports only the final, corrected results; how they were produced, including the reruns, is recorded in the master document (Section 8.21) and in results/eval_test_log.csv.

12. The evaluation, and how the numbers were made

12.1 What was tested, in one table

Phase 13 ran every experiment on the test split. Each answers one question:

Experiment	The question	Data	Short answer
Tactics	How well does DistilBERT find tactics, against simple keyword lists?	140 labelled real test emails	Not distinguishable from the keyword lists
Claims	How well does the extractor find claims?	The same 140 emails	Macro-F1 0.568 on the five common types
Score budget	How often does legitimate mail reach Suspicious?	All test email, per source	Within budget in 4 of 5 groups
N1	How much of a detector's accuracy is link-reading?	14,903 test emails, corpus labels	Measurably some, in all three model families
N2	Does the thread verifier catch what single-message checks miss?	25 Apache and 51 Enron takeover cases, plus the other variants	Yes, on these synthetic cases
N3	Does conditioning on the claim beat simpler uses of the same information?	3,045 attacks and 6,357 ham	No: it detects fewer attacks
Architecture	Does routing beat a flat classifier given the same signals?	The same	No for detection; yes for traceable reasons
Style confound	Can the collections be told apart by wording alone?	Pairs of sources	Yes, almost perfectly
Paraphrase	Does rewording an attack defeat it?	144 rewritten attack emails and 59 ham emails	Mostly kept; some new flags

12.2 The protocol: how the test was kept honest

A test score means something only if nothing was chosen after looking at the test data. Phase 13 used four safeguards.

1. The freeze record. Before the first test run, a script (src/eval/freeze.py) computed a SHA-256 hash of every frozen file: the patterns, the keyword lists, the verifier and thread rules, the score numbers and the reliability file, the preprocessing code, the tactic model and its thresholds, the N1 model weights, the claim settings, and the benchmark and label files, together with the version strings and the experiment settings. A hash is a fingerprint: change one character in a file and its hash changes completely. The record was committed to GitHub before the first test run (its own fingerprint begins 64e769a7b217cea6). Every test script recomputed the hashes and **refused to start** if anything differed.

2. The dress rehearsal. Every script runs on the validation split by default and needs an explicit flag to touch the test split. Each was first run on validation, which is where bugs in the evaluation code were found and fixed for free (for example, a chart that failed on summary rows). Nothing frozen changed afterwards.

3. The test log. Every test run wrote its start and finish to results/eval_test_log.csv. Ten test runs finished on 10 October 2026, none repeated. On 15 October, four scripts were rerun once each, with a --rerun flag and the reason written into the log, to fix the two reporting bugs described in Section 11.4. A rerun is allowed for a bug in an evaluation script, never to get a better number, and a comparison of the files before and after showed that every pooled number was unchanged.

4. Fixed settings. The claim settings were chosen on validation and written to a file before any test email was read; the keyword thresholds are the ones tuned on validation in Phase 6; both N1 models and every comparison system use fixed settings.

One honest exception. One check of the evaluation itself was changed after it failed in a rehearsal, before any test number existed. The N1 models were trained in half precision on Colab's GPU, and the check that the Mac reproduces Colab's probabilities used a tolerance of 0.001; it read 0.001193. The tolerance for half precision was set to 0.005 (a wrong model would differ by tenths, not thousandths), and a second check was added that no decision at the 0.5 cut changes unless Colab's probability was within that tolerance of 0.5 (none did, on 200 emails). It is a check of the evaluation, not a frozen component, and the master document records it.

How the uncertainty is shown. Every rate has a 95% interval from 1,000 bootstrap resamples (seed 42) that draw whole groups: subject groups for emails, threads for the hijack cases, pairs for synthetic emails. Differences between systems are paired. A cell with fewer than 10 positives is a count. Real and synthetic results are always apart.

12.3 Tactics: DistilBERT against the keyword lists

On the 140 labelled real test emails (labels from one model family), with thresholds fixed on validation:

Tactic	Positives	DistilBERT F1 [95%]	Keyword baseline F1 [95%]	DistilBERT on validation
Authority	25	0.526 [0.340, 0.667]	0.167 [0.000, 0.333]	0.593
Urgency	44	0.574 [0.452, 0.685]	0.557 [0.420, 0.691]	0.606
Scarcity	26	0.436 [0.250, 0.588]	0.632 [0.421, 0.791]	0.552
Secrecy	16	0.381 [0.117, 0.667]	0.645 [0.424, 0.846]	0.700
Macro-F1 (four tactics)		0.479 [0.370, 0.574]	0.500 [0.408, 0.581]	0.613

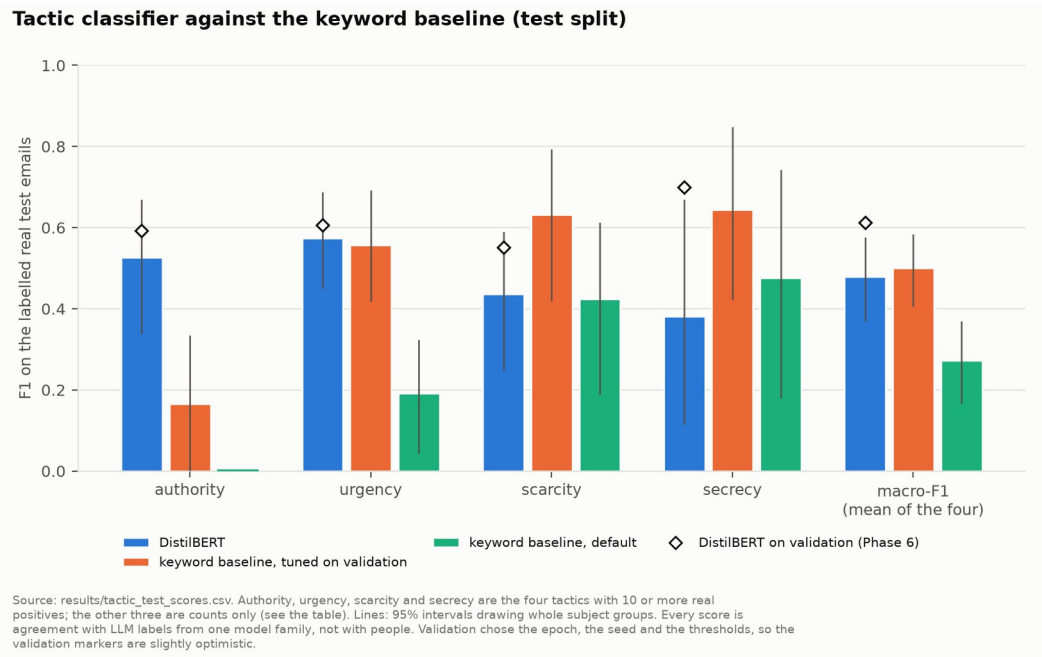


Figure 8. Tactic F1 on the test split (docs/figures/results/fig_tactics_test.png)

Reading it. The paired difference in macro-F1, DistilBERT minus the keyword baseline, is -0.021 [-0.144, +0.091]. The interval includes 0, so **neither is better**. DistilBERT is ahead on authority, the keyword lists on scarcity and secrecy, and they are close on urgency. The test macro-F1 (0.479) is lower than the validation one (0.613), which is expected: validation chose the epoch, the seed and the thresholds, so it was slightly optimistic. The three rare tactics are counts only: reciprocity and social proof had no real positives in test, liking had 4 (1 found, 2 false alarms, 3 missed).

Why this is not a disaster. The tactic classifier is a plain component, not a novelty, and its role in the score is deliberately small (at most 12 points, plus the multiplier on a contradiction already found). A tie with a good keyword list is an honest finding about a small labelled sample.

12.4 Claims: the extractor

On the same 140 emails, each claim type at the setting chosen on validation. The last column is how well the two labelling models agree with each other on that type: about the most the extractor could be expected to match.

Claim type	Positives	Found / false / missed	Test F1 [95%]	Labellers agree (F1)
affiliation_internal	12	2 / 2 / 10	0.250 [0.000, 0.533]	0.410
affiliation_external	44	33 / 23 / 11	0.660 [0.549, 0.757]	0.722
authority	16	14 / 9 / 2	0.718 [0.526, 0.870]	0.662
credential_request	15	10 / 6 / 5	0.645 [0.417, 0.815]	0.730
signature_contact (strong only)	26	15 / 12 / 11	0.566 [0.383, 0.704]	0.613

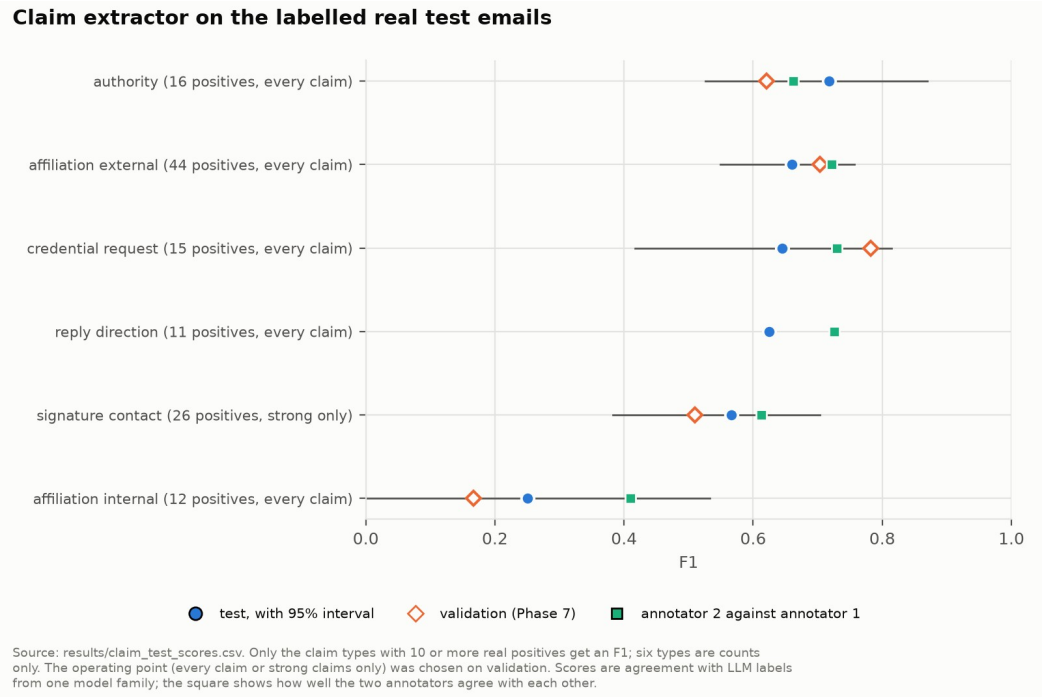


Figure 9. Claim extraction on the test split (docs/figures/results/fig_claims_test.png)

Reading it. Macro-F1 over these five types is 0.568 [0.470, 0.646]. Authority is the strongest. Internal affiliation is the weakest (2 of 12 found), and even the two labelling models agree on it at an F1 of only 0.410: "this is David from Finance" is easy, but many internal claims are implied rather than stated. Reply direction scored F1 0.625 on its 11 test positives but had too few validation positives to be one of the five types in the average. The other types have too few real positives and are counts (for example payment_request: 2 positives, 0 found, 10 false). A missing claim is never treated as evidence of honesty, which matters because a claim the extractor misses cannot be checked.

12.5 The risk score on the test split

How the frozen score sorted the test emails. Most emails have no claim that could be checked, so both denominators are shown:

Category	Emails	Suspicious or above	High risk	With a checked claim	Suspicious or above, of checked	High risk, of checked
Fraud	486	22.8%	6.8%	227	48.9%	14.5%
Phishing	2,559	11.8%	4.5%	561	54.0%	20.7%
Ham	6,357	0.3%	0.0%	398	4.3%	0.0%
Spam	5,501	0.2%	0.0%	116	9.5%	1.7%

Attacks and legitimate mail come from different collections with different headers, so these are descriptions, not precision or recall.

The false-alarm budget (at most 1% High risk and 5% Suspicious or above among checked legitimate emails per source, judged where at least 20 were checked), measured on the test split, which is the clean measurement:

Group	Checked	Suspicious or above	High risk	Verdict
Ham, Apache Kafka	17	11.76%	0.00%	Fewer than 20: information only
Ham, Apache Tomcat	17	0.00%	0.00%	Fewer than 20: information only
Ham, CEAS-08	268	5.60%	0.00%	**Over**
Ham, SpamAssassin	96	0.00%	0.00%	Within
Real thread messages, Apache Kafka	95	0.00%	0.00%	Within
Real thread messages, Apache Tomcat	201	1.00%	0.00%	Within
Real thread messages, Enron	633	4.90%	0.47%	Within

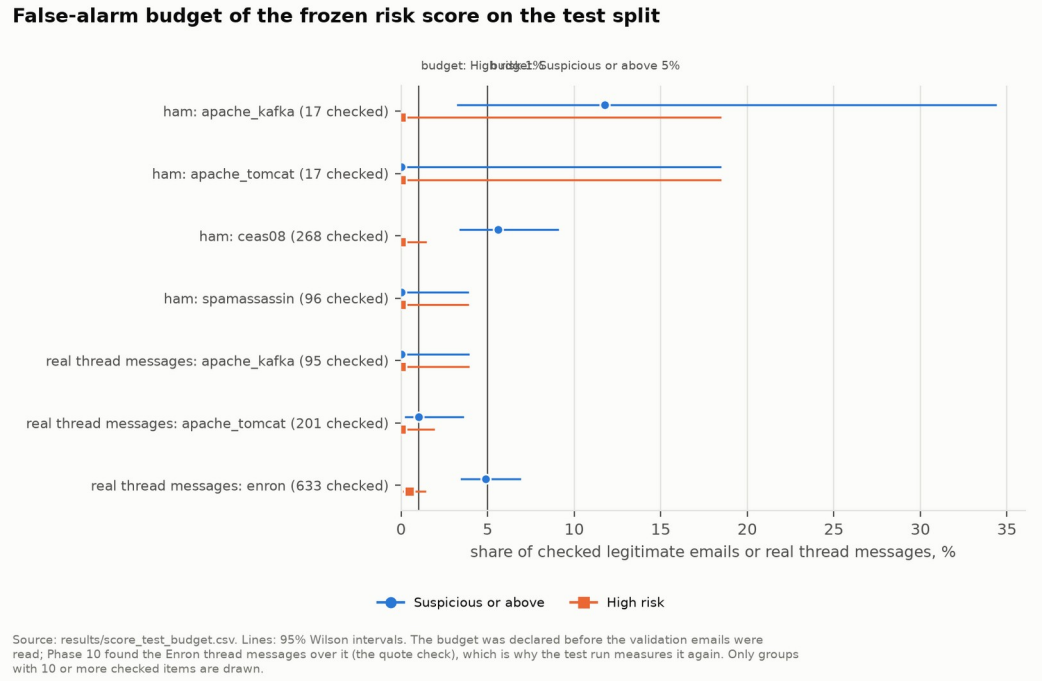


Figure 10. The false-alarm budget on the test split (docs/figures/results/fig_score_budget_test.png)

Reading it. Within budget in 4 of 5 judged groups. CEAS-08 ham was over, at 5.60% of 268 checked emails, with a 95% interval of [3.42%, 9.03%] that still includes the 5% budget. The miss is reported, not fixed: changing the score after seeing the test split would have destroyed the test.

12.6 N1: how much of a detector's accuracy is link-reading?

Attack (phishing, fraud) against not-attack (ham, spam), on the test emails, with corpus labels. Model A trained on raw bodies, model B on redacted bodies, from one training sample of 9,856 emails with two seeds and a fixed threshold of 0.5. Attack-class F1 [95% interval]:

Model	Raw view	Redacted view	Link-free, raw	Link-free, redacted
DistilBERT A (trained raw)	0.935 [0.927, 0.943]	0.877 [0.864, 0.889]	0.863 [0.840, 0.884]	0.854 [0.830, 0.876]
DistilBERT B (trained redacted)	0.918 [0.908, 0.927]	0.932 [0.923, 0.940]	0.862 [0.839, 0.884]	0.861 [0.837, 0.883]

Model	Raw view	Redacted view	Link-free, raw	Link-free, redacted
TF-IDF A, same sample	0.908 [0.897, 0.918]	0.870 [0.857, 0.882]	0.851 [0.826, 0.874]	0.845 [0.819, 0.869]
TF-IDF B, same sample	0.885 [0.873, 0.896]	0.883 [0.871, 0.893]	0.845 [0.820, 0.868]	0.843 [0.817, 0.867]
TF-IDF A, all train	0.931 [0.921, 0.939]	0.896 [0.885, 0.907]	0.873 [0.850, 0.895]	0.868 [0.845, 0.890]
TF-IDF B, all train	0.914 [0.903, 0.924]	0.908 [0.897, 0.917]	0.870 [0.847, 0.891]	0.869 [0.845, 0.890]
Link-presence rule	0.413 [0.385, 0.440]		0.000	

The quantity that answers the question is **A's drop minus B's drop** (the drop is F1 on raw minus F1 on redacted):

Family	A's drop	B's drop	A's drop minus B's drop
DistilBERT	+0.059 [+0.050, +0.069]	-0.014 [-0.019, -0.009]	**+0.073 [+0.061, +0.086]**
TF-IDF, same sample	+0.038 [+0.031, +0.045]	+0.003 [-0.004, +0.008]	+0.035 [+0.025, +0.046]
TF-IDF, all train	+0.035 [+0.029, +0.041]	+0.006 [+0.001, +0.011]	+0.028 [+0.021, +0.037]

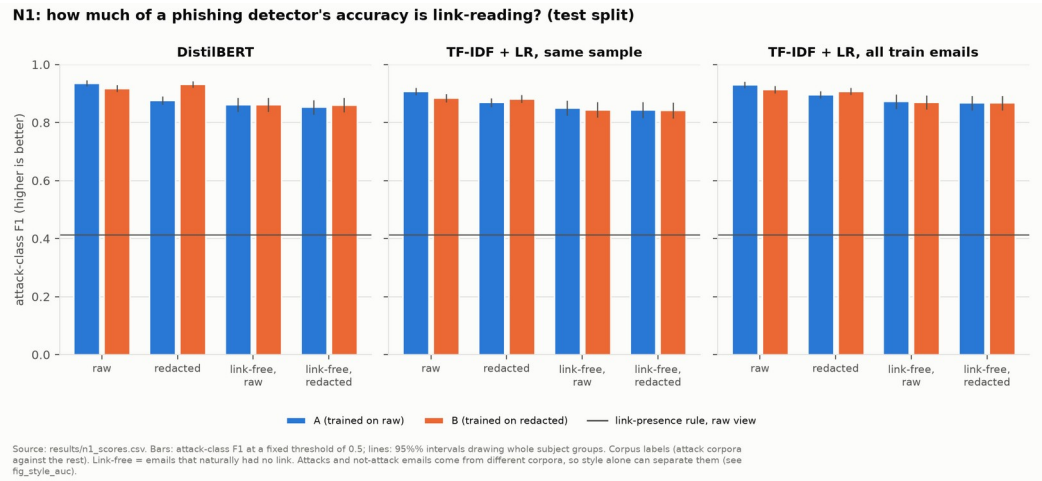


Figure 11. N1: F1 on each view (docs/figures/results/fig_n1_ablation.png)

Where the loss sits. DistilBERT A's recall per attack source, raw against the link-free redacted view: phishing_pot 94.5% (1,058 of 1,120) to 78.6% (140 of 178); Nazario 94.1% (1,354 of 1,439) to 82.5% (193 of 234); Nigerian fraud 99.4% (483 of 486) to 99.3% (282 of 284). Fraud emails rarely depend on links; phishing does.

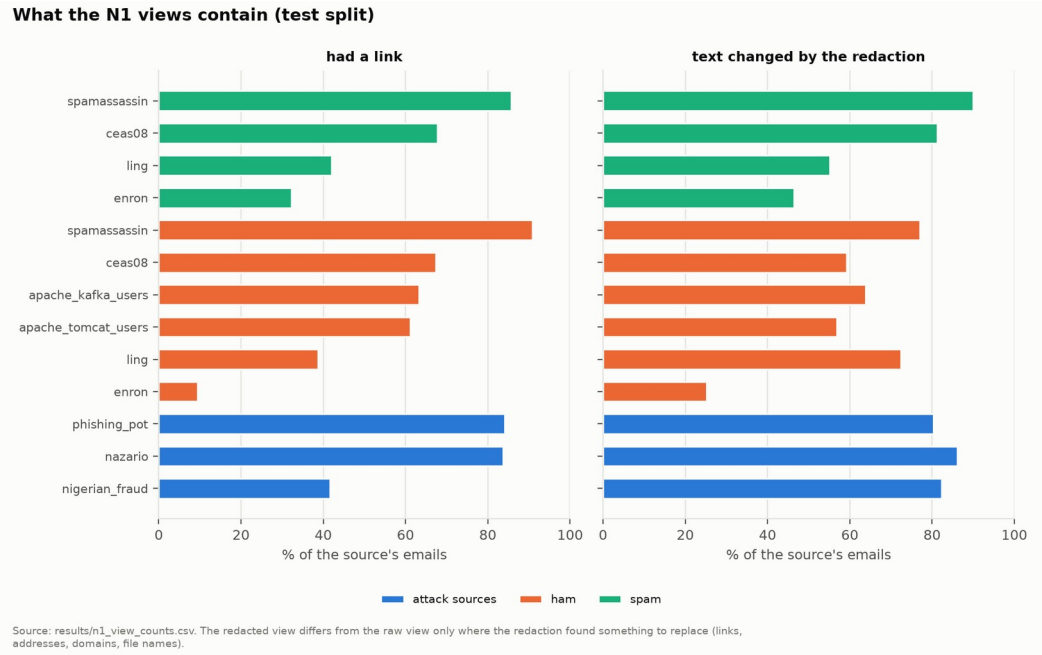


Figure 12. N1: recall per source (docs/figures/results/fig_n1_links.png)

Reading it.

- A loses accuracy when links are hidden; B does not (B even gains a little, because it was trained on that view). The difference is above 0 in all three families, so **part of the accuracy of a raw-trained detector is link-reading**.
- Training on redacted bodies keeps most of the accuracy: B on redacted text scores 0.932, against 0.935 for A on raw text.
- On the 696 test attacks that never had a link, A and B are not distinguishable (0.863 and 0.862). But that subset is a different mix of sources, so it is not a before-and-after of the same emails.
- A rule that just flags every email with a link reaches F1 0.413, with a false positive rate of 46.5% on ham: links alone are a poor detector, but a model can still lean on them.

12.7 N2: does the thread verifier catch what single-message checks miss?

The test hijack cases: real base threads, synthetic injected messages and headers. "Detected" means a medium or high contradiction on the injected message.

Source	Case	Cases	Thread verifier (N2)	Header and request verifiers alone	Scan flips at the injected message
Apache	A, takeover	25	72.0% [52.0%, 88.0%] (18)	20.0% (5)	68.0% (17)
Apache	B, lookalike swap	17	100.0% (17)	0.0% (0)	100.0% (17)
Apache	C, forged thread	25	100.0% (25)	0.0% (0)	96.0% (24)
Apache	Real next reply (false alarms)	25	0.0% (0)	0.0% (0)	
Enron	A, takeover	51	94.1% [88.2%, 100.0%] (48)	0.0% (0)	78.4% (40)

Source	Case	Cases	Thread verifier (N2)	Header and request verifiers alone	Scan flips at the injected message
Enron	C, forged thread	51	88.2% [78.4%, 96.1%] (45)	0.0% (0)	70.6% (36)
Enron	Real next reply (false alarms)	51	2.0% (1)	0.0% (0)	

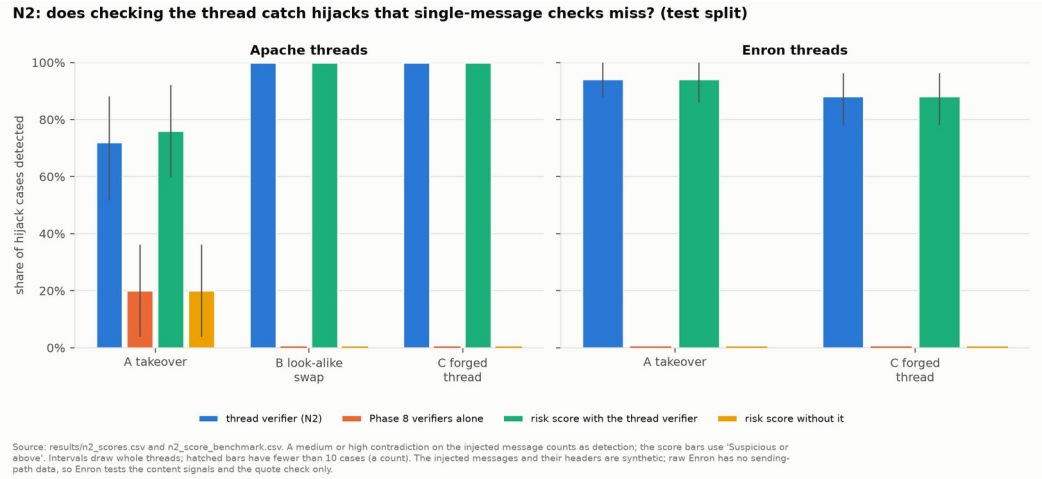


Figure 13. N2: hijack cases detected (docs/figures/results/fig_n2_hijack.png)

False alarms on real, unmodified test threads (messages that have earlier messages): 0.00% of 95 Kafka messages, 0.50% of 201 Tomcat messages, and 4.44% [3.08%, 5.99%] of 698 Enron messages.

Reading it.

- **Variant A is the real test.** A takeover that changes only the content: the thread rules caught 18 of 25 Apache and 48 of 51 Enron cases, against 5 of 25 and 0 of 51 for the single-message checks.
- **Variants B and C** each change exactly one thing the rules are defined to check (the sending path; the IDs and quotes), so their near-perfect detection mainly confirms that the rules work as specified.
- **It points to the right message** most of the time: the first contradiction falls on the injected message in 17 of 25 Apache and 40 of 51 Enron takeover cases.
- **It is quiet on real conversations:** 0 of 25 and 1 of 51 false alarms on the real next reply.
- **But** the injected messages and their headers are synthetic and written from the same fields the signals read, so these rates test the rules against the construction, not against real attackers. Enron's 4.44% false alarms on real messages is the honest measure of noise.

12.8 N3: does conditioning on the claim beat the simpler alternatives?

The frozen claim-routed score against learned systems (logistic regressions fitted on the train split, attack against ham) that use the same information more simply:

- **Text only:** tactic probabilities and claim counts.
- **Headers only:** authentication verdicts, free mailbox, lookalike, Reply-To and similar fields.
- **Parallel fusion:** the average of the text score and the header score.

- **Flat classifier** (the architecture comparison, Section 12.9): every ledger signal in one list, without the link between a claim and its evidence.

To compare fairly, each learned system is held to the same false-alarm rate the routed score shows on validation ham (0.14%), with a floor of 0.5% so that the cut is not set by a handful of emails.

System	F1 [95%]	Recall	False alarms (ham)	Precision	AUC
Routed score	0.238 [0.204, 0.277]	13.6%	0.3%	0.961	0.812
Text only	0.639 [0.612, 0.666]	47.4%	0.5%	0.979	0.949
Headers only	0.788 [0.767, 0.810]	65.5%	0.3%	0.990	0.966
Parallel fusion	0.882 [0.869, 0.895]	80.3%	0.8%	0.980	0.993
Flat classifier	0.658 [0.629, 0.687]	49.7%	0.5%	0.977	0.949

(3,045 test attacks against 6,357 test ham.)

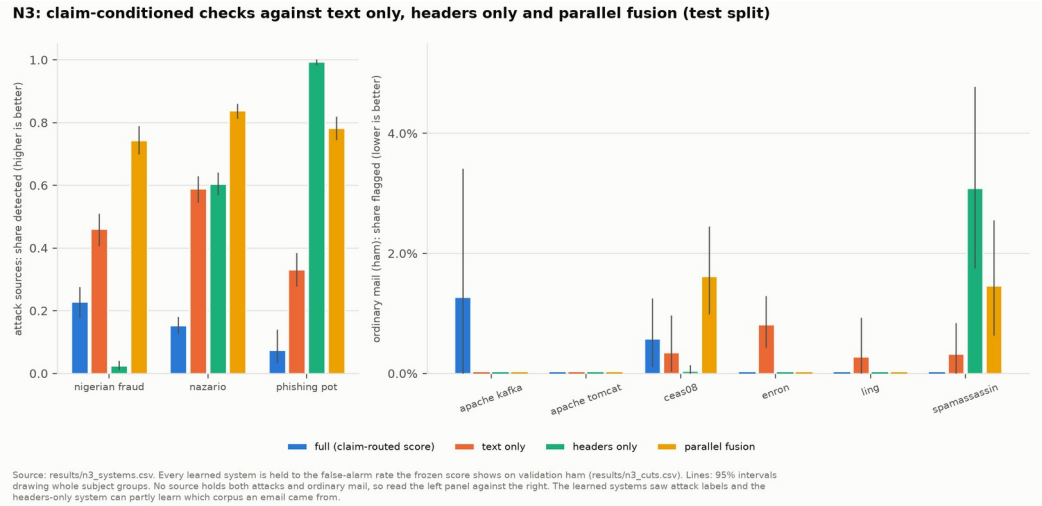


Figure 14. N3: detection by system (docs/figures/results/fig_n3_detection.png)

All the paired differences lie below 0. For example, routed minus parallel fusion is -0.644 [-0.678, -0.605] in F1. AUC, which needs no cut, is also the lowest for the routed score (0.812 against 0.949 to 0.993).

Per source. Share flagged on the test split (attack rows: detected; ham rows: false alarms; spam rows: flagged, though no system was fitted on spam):

Source	Emails	Routed	Text	Headers	Fusion	Flat
Nigerian fraud	486	22.8%	46.1%	2.5%	74.3%	52.9%
Nazario	1,439	15.3%	58.9%	60.4%	83.9%	58.3%
phishing_pot	1,120	7.4%	33.2%	99.5%	78.3%	37.1%
Apache Kafka (ham)	158	1.3%	0.0%	0.0%	0.0%	0.0%
Apache Tomcat (ham)	324	0.0%	0.0%	0.0%	0.0%	0.0%
CEAS-08 (ham)	2,588	0.6%	0.4%	0.0%	1.6%	0.3%
Enron (ham)	2,313	0.0%	0.8%	0.0%	0.0%	1.0%
Ling (ham)	359	0.0%	0.3%	0.0%	0.0%	0.3%

Source	Emails	Routed	Text	Headers	Fusion	Flat
SpamAssassin (ham)	615	0.0%	0.3%	3.1%	1.5%	0.5%
CEAS-08 (spam)	3,126	0.0%	2.1%	0.1%	10.2%	2.0%
Enron (spam)	2,055	0.0%	8.6%	0.0%	0.0%	8.0%
Ling (spam)	69	0.0%	7.2%	0.0%	0.0%	7.2%
SpamAssassin (spam)	251	4.0%	12.3%	25.5%	24.7%	15.5%

Look at the headers-only column: 99.5% of phishing_pot but 2.5% of Nigerian fraud. phishing_pot carries modern authentication headers and the fraud collection kept none, so a headers-only model is partly learning **which collection** an email came from. That is the corpus confound of Section 2.8, visible in one column.

The synthetic colleague-impersonation arm. 31 pairs of synthetic emails with clearly synthetic header blocks written by rule: the attack from a free mailbox or a lookalike domain that passes its own authentication, its twin from the company's own domain.

System	Attacks flagged	Twins flagged (false alarms)
Routed score	19 of 31 (61.3% [45.2%, 77.4%])	0 of 31
Text only	11 of 31	0 of 31
Headers only	31 of 31	31 of 31
Parallel fusion	17 of 31	4 of 31
Flat classifier	17 of 31	0 of 31

Headers only flags every attack and every honest twin: it cannot tell a colleague's real email from an impersonation, because it does not know what the email claims. The routed score is the only system here that flags most attacks while flagging no twin. But the text and the headers of this arm are synthetic, so it shows what the design can do on the case it was built for, not how often that case occurs.

Reading it, plainly. The claim-routed score **does not detect more attacks** than the simpler systems on these public collections; it detects fewer. Three things limit the comparison, and none of them removes the gap:

1. The learned systems were fitted on attack labels; the routed score uses no attack labels at all.
2. A headers-only model can partly learn which collection an email came from (the column above).
3. The 0.5% floor lets each learned system flag more validation ham than the routed score does (0.14%), so AUC, which needs no cut, is the fairer comparison, and it gives the same answer.

Part of the reason is simple: the routed score can only raise an email to Suspicious if the email has a claim that could be checked, and only 788 of the 3,045 test attacks have one (398 of the 6,357 ham do). On those 788, its F1 is 0.679 [0.633, 0.734], and it is not distinguishable from text only (difference -0.063 [-0.136, +0.026]) or headers only (-0.057 [-0.119, +0.003]), though still behind fusion and the flat classifier.

12.9 The architecture: routed against flat

	Routed	Flat
F1	0.238 [0.204, 0.277]	0.658 [0.629, 0.687]
Flagged emails on the test split	442	1,819
Flags with a claim and a rule behind them	442 of 442	0 of 1,819

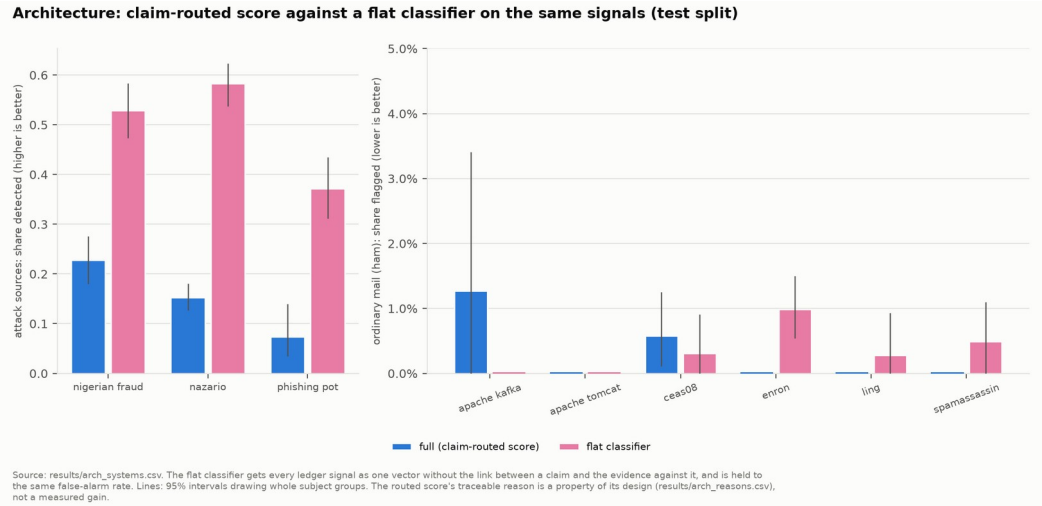


Figure 15. Routed against flat (docs/figures/results/fig_arch.png)

Where they disagree (emails flagged by both, only by the routed score, only by the flat classifier, by neither): fraud 102 / 9 / 155 / 220; ham 6 / 11 / 29 / 6,311; phishing 271 / 32 / 984 / 1,272; spam 7 / 4 / 265 / 5,225.

Reading it. The flat classifier detects more. The routed design is the one where every alarm names its claim, its rule and its evidence. That is a **property of the design**, which the code checks on every report, **not a measured gain**: nobody tested whether security analysts work better with it.

12.10 The style-confound test

How well can a simple model tell two collections apart from their words alone (AUC; 0.5 is chance, 1.0 is perfect)?

- Two sources of the same kind (two ham sources, two spam sources, two attack sources), by their words: median AUC **0.996** over 24 pairs (range 0.937 to 1.000).
- The same pairs by their tactic probabilities: median 0.693 (range 0.509 to 0.990).
- Real against synthetic attack emails, by words: **1.000** [1.000, 1.000] (90 real and 31 synthetic test emails); by tactic probabilities 0.956 [0.889, 1.000].
- Real against synthetic benign emails, by words: 1.000 [1.000, 1.000]; by tactic probabilities 0.945 [0.882, 0.986].

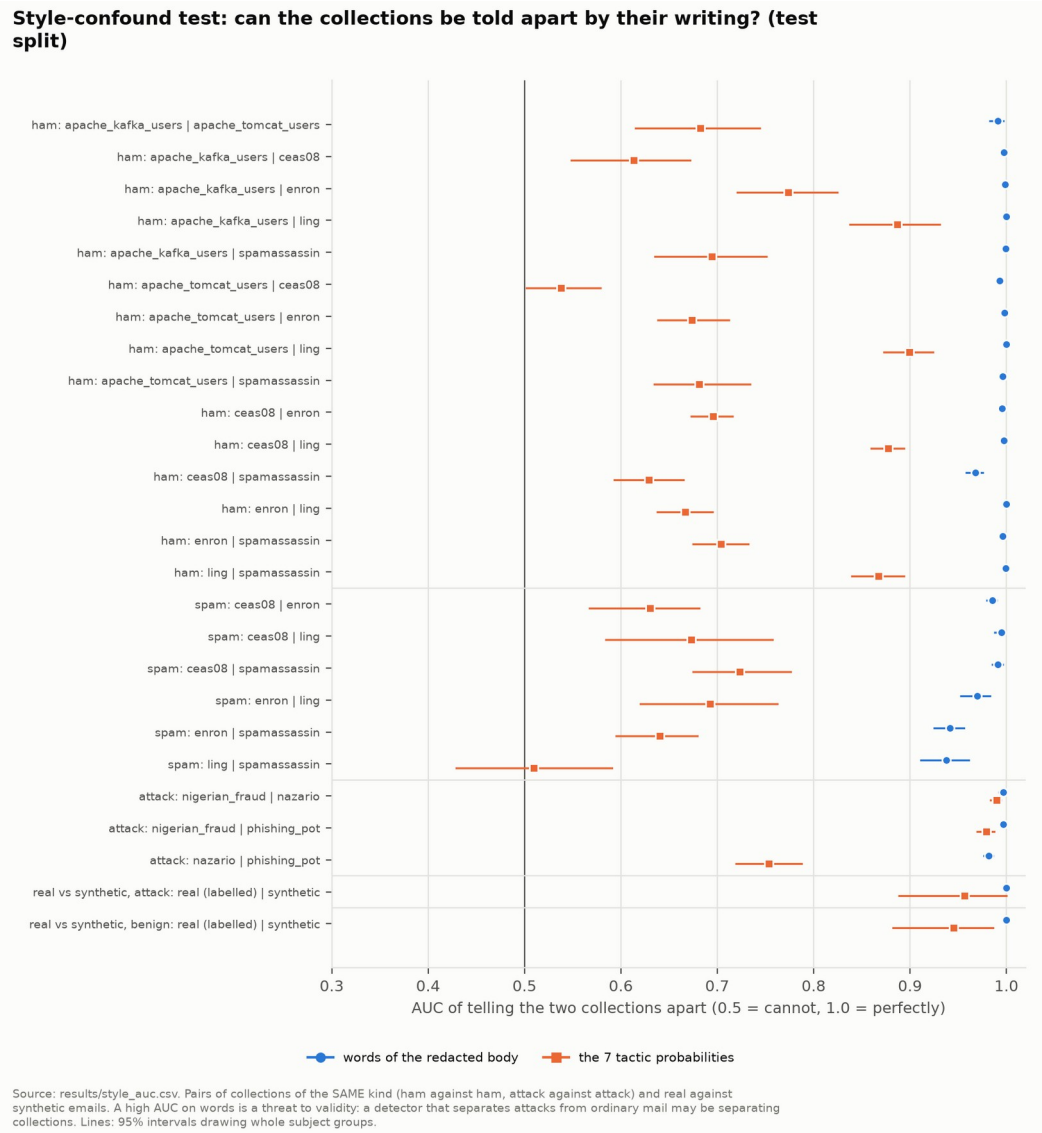


Figure 16. Style-confound test (docs/figures/results/fig_style_auc.png)

Reading it. Every collection has a recognisable style, and synthetic emails are perfectly recognisable. This is not a defect of PretextGuard; it is a property of the data that every result must be read against. It is why results are reported per source, why synthetic results are always kept apart, and why no pooled number is read as "detection" alone.

12.11 The paraphrase test

A language model rewrote the redacted body of sampled test emails (the headers were not touched), and the real analyze() ran before and after. The ham group is a control for new false alarms. Meaning was checked by hand on six pairs only.

Group	Valid rewrites	Flagged before	Flagged after	Kept	New flags
Ham (control)	59 of 60	0	1	0 of 0	1 of 59
Nigerian fraud	50 of 50	11	12	9 of 11	3 of 39
Nazario	49 of 50	10	9	6 of 10	3 of 39
phishing_pot	45 of 50	3	3	3 of 3	0 of 42

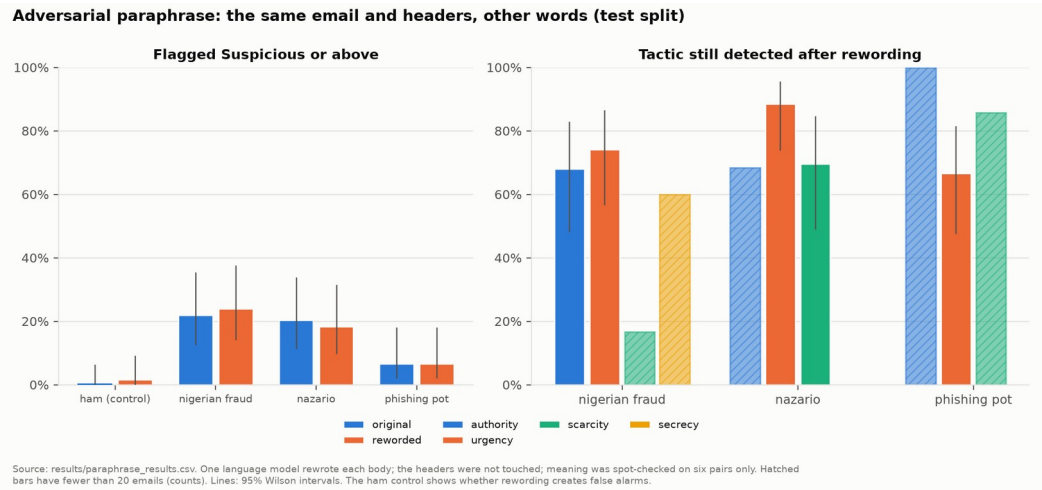


Figure 17. Paraphrase test (docs/figures/results/fig_paraphrase.png)

Reading it. Rewording kept 18 of the 24 flags the attack groups started with and added 6 new flags among 120 attack emails not flagged before; the ham control gained 1 flag among 59. The headers were untouched, so a header contradiction survives rewording by construction; the test mostly shows that the claim extractor and the tactic model are not easily fooled by a simple rewrite of these emails.

12.12 What it all honestly shows

1. **Detection, pooled.** The claim-routed score is not the best detector on public collections; four simpler learned systems find more attacks. The comparison favours them in known ways, but the gap is large and real.
2. **Traceability.** Every one of its 442 alarms names a claim, a rule and the evidence. That is a design property, not a measured benefit.
3. **Threads (N2).** On synthetic takeover replies in real threads, the thread rules catch what single-message checks miss, and they are quiet on real conversations. The cases are synthetic.
4. **Links (N1).** Part of a raw-trained detector's accuracy is link-reading, in all three model families, and training on redacted text costs little.
5. **Tactics.** DistilBERT and good keyword lists are not distinguishable on 140 labelled test emails.
6. **Score budget.** Within budget in 4 of 5 groups; one miss, reported.
7. **Confounds.** The collections, and real against synthetic emails, are separable by wording alone; every contrast between collections must be read with that in mind.
8. **Rewording.** Most flags survive a language-model paraphrase of the body.

12.13 Two sets of numbers people mix up

There are two very different kinds of numbers in this project.

- **Checks** (PASS or FAIL) test that the **code** does what it should: "this IBAN must appear masked", "a fake dmarc=pass must be ignored", "the API must refuse a request without the key". They take seconds, use hand-made inputs, and must all pass: the API's 88, the website's 96 and 93, the evaluation's self-test of 110, and so on. They say nothing about how well the system detects attacks.

- **Results** (F1, recall, AUC, false-alarm rates with intervals) measure how the **whole system** behaves on data. They are what the paper reports, and every one of them comes from a file in results/.

12.14 The limits of all this

- Tactic and claim scores are agreement with labels from one family of language models; no person validated them. The N1 labels are the collections' own labels.
- No source holds both attacks and ordinary mail, so every pooled number mixes "which collection" with "attack or not".
- The learned comparison systems saw attack labels and the routed score did not; a headers-only model can partly learn the collection. Both favour the learned systems.
- The hijack cases, the synthetic header blocks and the paraphrases are machine-written: they show that the rules do what they are defined to do, not how often real attackers behave that way.
- N1 rests on one training sample of 9,856 emails and two seeds per model, so its intervals cover the test sample, not the randomness of training.
- The paraphrase test used one language model and 50 emails per attack source; its meaning check was six pairs by hand.
- Everything was frozen before the test split was read; where a result is unwelcome, it is reported as it is and the component was not changed.

13. The paper

13.1 What it is

- **Title:** Detection of Payload-Free Pretexting Emails Through Claim-Routed Verification Against Email Headers and Thread History.
- **Format:** an IEEE-style conference paper, 6 pages, A4, two columns, built in the IEEE template. Files: docs/paper/Paper_Draft.docx and Paper_Draft.pdf (draft 4, 15 October 2026), with a text copy in the claude.ai project.
- **Name of the system in the paper:** "the claim-routed verification pipeline", or "the pipeline". The word PretextGuard and the repository are not named anywhere in the paper; the project and the repository keep their name.
- **Use:** your faculty evaluates the project from this paper. The venue, the author names and whether review is blind were left as placeholders.

13.2 How it is organised

Section	What it says
Abstract	The problem, the idea, the three headline results and the plain statement that the pipeline is not the best pooled detector
I. Introduction	Business email compromise and pretexting with the FBI and Verizon figures; the David example; why detectors miss it; the problem framed as fact-checking; four contributions
II. Related Work	Persuasion and tactics; identity from metadata and history; claims and model judgement; each paragraph ends with how the pipeline differs
III. Problem and Threat Model	Payload-free email; the outside impersonator; the attacker inside a real account; which headers are trusted; what is out of scope
IV. The Pipeline	Fig. 1 (the pipeline), redaction, claims and tactics, routing and verification, the ledger and the score, Table I (the ledger of the David example), one short paragraph on implementation
V. Evaluation Design	The data, the threads, the labels, the three experiments, the protocol
VI. Results	Table II (pooled detection, the hijack cases, the link experiment), Table III (every system per source), then the tactic, claim and score-budget results
VII. Discussion and Limitations	What the results support and what they do not, and every limit
VIII. Conclusion	One paragraph
References	28, in IEEE style

The paper's four contributions: (1) the claim-routed verification pipeline; (2) the payload-free evaluation protocol, N1; (3) thread consistency verification, N2; (4) an evaluation with frozen components, a test log, grouped bootstrap intervals and per-source reporting, which also shows that the pipeline is not the best pooled detector on public corpora.

13.3 What it argues

In one paragraph: payload-free pretexting is invisible to scanners and often passes authentication. Treating it as fact-checking (claims in the body, evidence in the headers and the thread) gives every alarm a stated reason and catches synthetic takeover replies that single-message rules miss. A fair test of link redaction shows that part of a normal detector's accuracy is link-reading. On pooled detection over public collections, however, the pipeline finds fewer attacks than simpler learned systems, so the design does not by itself detect more attacks on these data. Testing it on real attacks needs data that does not yet exist publicly: labelled payload-free business email compromise, real hijacked threads with their authentication headers, and a sample labelled by people.

13.4 The closest earlier work, in plain words

The paper cites only works that were read on a free page, for exactly what they say. The ones that matter most:

Work	What they did	How the pipeline differs
Paul et al. (2024)	Labelled 940 emails with the warning signs users are trained to notice (including the organisation a sender claims) and trained a classifier per sign; observed that a mismatch between a claim and the metadata is often a stronger signal than the content, and assumed "a downstream verification algorithm" would do that comparison	The pipeline is such an algorithm
BEC-Guard, Cidon et al. (2019)	A commercial detector: decides from an organisation's header history whether a sender impersonates an employee, then classifies the body and its links after removing names and signatures; 98.2% precision on more than 4,000 attacks; account takeover outside its focus	Because names and signatures are removed, no step compares a claim with the headers; the pipeline needs no mail history beyond the thread
ChatSpamDetector, Koide et al. (2024)	Asks GPT-4 to look for brand impersonation and header spoofing; 99.70% accuracy on 2,010 emails, but every email without a link was excluded, and each email costs one model call	So the paper does not claim to be first to compare a claimed sender with the headers; the pipeline uses fixed rules with recorded reasons, no model at run time, and is evaluated on link-free text
Ho et al. (2019)	Phishing sent from compromised accounts across 92 organisations; their detector needs a malicious link, and they say their data likely underestimate attacks without one	Motivates the payload-free focus and N2
Gascon et al. (2018)	Profile senders from email structure without reading text; 90% of spoofed emails at fewer than one false alarm in 10,000; note that such features cannot detect a hacked account	That hacked-account case is what N2 addresses

Work	What they did	How the pipeline differs
Karki et al. (2022), Tooher and Lallie (2025), Pan et al. (2025)	Persuasion in phishing text: transformer classifiers on automatic labels; a detector trained on machine-written attacks against real legitimate mail; a large study finding authority cues in 27.10% of legitimate samples	All read the body only; machine-written attacks against real mail let writing style separate the classes; tactics alone are weak evidence
ConvoSentinel, Ai et al. (2024)	Detects social engineering across whole chat conversations	Chat, with no headers or thread metadata
Guo et al. (2022)	Survey of automated fact-checking: claim detection, evidence retrieval, claim verification	The pipeline follows this division
Arp et al. (2022); Geirhos et al. (2020); Pangakis et al. (2023)	Pitfalls of machine learning in security; shortcut learning; language-model labels need checking against human labels	Frame the evaluation and its limits

13.5 What the paper may and may not claim

May claim, always with the sample stated: the design and how it works; 18 of 25 and 48 of 51 synthetic takeover cases against 5 and 0; the link-reading result in three model families; 442 of 442 alarms with a claim, a rule and evidence; the tactic tie; the score budget in 4 of 5 groups; every limitation.

May not claim: that the pipeline is the best detector (it is not, on these data); that it is the first to compare a claimed sender with the headers (ChatSpamDetector already does something close); that no thread-hijacking detector exists anywhere (the paper says "we did not find an academic detector with an open evaluation"); that BEC-Guard adds two scores (it is a gated pipeline); that Paul et al. built no detector (they trained classifiers); any general accuracy figure; any result on real attackers for the thread checks; that the labels are human labels.

Wherever an interval includes 0, no system is called better.

13.6 How it was written

- **Sources:** every reference was found on a free page and at least its abstract read; the close neighbours were read in full. The source table and the literature notes (docs/paper/) record how each was checked and what it may support.
- **Numbers:** every number in the paper comes from a file in results/ or from master document Section 8.21. A script extracted all 261 distinct numbers from the paper and matched each one against those sources.
- **Final results only:** the paper reports the final results. How they were produced, including the logged reruns of 15 October, is recorded in the master document and the test log, not in the paper.
- **Style:** formal and plain, no em dashes, hyphens only in names and fixed terms, a final pass to remove phrasing that sounds machine-written.

14. What it can and cannot do

It can:

- Analyse one email (pasted text or a .eml file, with or without headers) or a whole thread, and return a 0 to 100 score, a band, a recommended action and a list of findings, each with its claim, rule, evidence and reason.
- Find eleven types of claim in English text and check each against the evidence that can test it.
- Catch an outside impersonator: a free mailbox or a lookalike domain claiming to be from your company or a known brand, a forged copy of your own domain that fails DMARC, a Reply-To that sends answers elsewhere, a signature address that differs from the sender.
- Catch changes inside a thread: new urgency or secrecy, new bank details or requests, a new sending path, reply IDs or quotations that do not match the real conversation; and point to the message where the thread flipped.
- Recognise when it cannot check something, and say so instead of guessing ("2 claims found, 1 could not be checked").
- Highlight the words behind each tactic it detects.
- Do all of this without storing the email and without calling any language model.

It cannot:

- See an attacker who has read the mailbox and copies the real reply IDs, the sending details and the calm tone of the real sender. No rule here can catch that message.
- Judge a claim it did not find. The extractor reads English only and misses paraphrases its patterns do not expect; affiliation_internal is its weakest type (2 of 12 found in the test).
- Check internal affiliation without knowing the recipient's company domain. Most public collections do not reveal it.
- Prove that an authenticated sender is honest: "consistent" means only that nothing contradicts the claim.
- Score a pasted body without headers on anything but tactics (at most 12 points), so a classic fraud body pasted without headers stays Low risk, with a note that nothing could be checked.
- Know about brands it was not told about: 51 organisations' domains are listed by hand.
- Serve many users over the internet as it stands: it is built as a single-user demo on one machine (Part 10).
- Claim measured performance on real business email compromise or real hijacked threads: no such public data exist, and the thread results come from synthetic cases.

14.1 The limits in one list

1. **Labels:** tactic and claim labels come from language models of one family, with moderate agreement and no human check.
2. **Data:** attacks and legitimate mail come from different collections, which differ in style and headers, so every pooled number mixes collection and attack.
3. **Synthetic parts:** the hijack cases, the colleague-impersonation headers, the rare-tactic emails and the paraphrases were written by models or by rule.
4. **Small samples:** 140 labelled test emails; 25 Apache and 51 Enron takeover cases; 31 synthetic impersonation pairs; one N1 training sample of 9,856 emails with two seeds.

5. **Detection:** on pooled public data, simpler learned systems find more attacks.
6. **Language:** English only.
7. **Deployment:** never tested at a real company.

15. How to explain it to someone

15.1 In 30 seconds

"The most expensive email fraud has no link and no attachment, so scanners miss it, and it often passes every authentication check. My project, PretextGuard, reads what an email claims about itself, like 'this is David from Finance' or 'our bank has changed', and checks each claim against the evidence that can prove it: the email's headers and its own conversation history. When the claim and the evidence disagree, that is the alarm, and it comes with the reason."

15.2 In 2 minutes

Add three things to the 30-second version.

- **The key insight:** SPF, DKIM and DMARC prove which domain sent an email, not who the person is. A real Gmail account passes all three, so "I am your finance director" from Gmail passed authentication for the wrong domain. And an attacker inside a real account passes everything, so the only evidence left is the conversation: suddenly urgent, suddenly secret, suddenly a new bank account.
- **How it works:** it extracts typed claims with rules, sends each claim to the checker that can test it (header checks, request checks, thread checks), records every verdict in a ledger with its reason, and turns the ledger into a 0 to 100 score. There is a fine-tuned DistilBERT for persuasion tactics, but tactics only make a found contradiction weigh more; they never create an alarm on their own.
- **The honest results:** the thread checks caught 48 of 51 synthetic takeover replies in real Enron threads where single-message checks caught none; a fair test showed that part of a normal detector's accuracy is just reading links; but as a plain detector on public data, simpler machine-learning models found more attacks, and I report that openly.

15.3 In 10 minutes

Use this order; each step points to the part of this document that covers it.

1. **The attack** (2.5): pretexting and business email compromise; the David email; payload-free; FBI figures.
2. **Why existing tools miss it** (2.4, 13.4): payload scanners; authentication proves the domain, not the person; BEC-Guard removes the names before reading the body; ChatSpamDetector excluded emails without links.
3. **The idea** (3): the immigration officer; conditioning on the claim; three kinds of verdict.
4. **The two attackers** (2.5): outside impersonator (N3), attacker inside the account (N2).
5. **The pipeline** (4): parser, redaction, claims, tactics, router, three verifiers, ledger, score; show the David ledger (5.1).
6. **The thread demo** (5.2): Marta's hijacked fifth message passes every header check, and the thread flips at message 5.
7. **The evaluation** (12): frozen before the test; N1 link-reading; N2 on the hijack benchmark; N3's unwelcome result and why it is reported; the style confound.
8. **Limits** (14): model labels, synthetic cases, different collections, English only.

15.4 Questions people are likely to ask

"Hasn't this been done already?" Parts of it. A 2025 paper detects persuasion in emails, but trains on machine-written attacks against real legitimate mail and reads the body only. ChatSpamDetector asks GPT-4 to look at the headers too, but excluded every email without a link and costs one model call per email. Paul et al. said a claim-metadata mismatch is a strong signal and left that comparison to "a downstream verification algorithm". PretextGuard is such an algorithm, adds thread history, and is evaluated on link-free text.

"If the attacker uses a real account, SPF passes. How do you catch them?" With the thread. A real account's new message is compared with the earlier messages of the same conversation: new urgency, new bank details, a different mail program or server, or reply IDs and quotes that do not match.

"How is this different from combining a text score and a header score?" That is parallel fusion, which the evaluation includes. Fusion averages two scores, neither of which knows what the other is about. PretextGuard checks each claim against the evidence for that claim, so it can say "claims Acme Finance; authenticates only as gmail.com". In the evaluation, fusion detected more attacks; PretextGuard gave a reason for every alarm and, on the synthetic impersonation pairs, flagged attacks without flagging their honest twins.

"Your detector scored lower than the simple models. Isn't the project a failure?" No. The project tested a hypothesis and reported the answer: on these public collections, conditioning on claims did not detect more attacks. Three known factors favour the simpler models (they saw attack labels; a header model can recognise the collection; the cut lets them flag more legitimate mail), but the gap is real. What the design does give, measurably, is a stated reason for every alarm, and the thread and link results stand on their own. Reporting an unwelcome result honestly is part of the work.

"Why not just use a large language model?" Cost and delay at the scale of a mail gateway, no way to run it inside the company's own network, different answers on different runs, and no inspectable reasoning. Language models were used only offline, to make labels and test cases.

"How were the labels made, and can you trust them?" Two Gemini models labelled 690 real emails separately at temperature 0, and a third broke ties. Mean Cohen's kappa was 0.501 on tactics and 0.458 on claims, which is moderate. They are one model family, and no person checked them, so every tactic or claim score means agreement with these labels, not correctness. That is stated wherever such a score appears.

"Isn't your data synthetic?" The main data are real: 99,324 emails from nine public collections, including real phishing and fraud. Synthetic emails fill only measured gaps (rare tactics, colleague impersonation, hijacked replies), always have a benign twin from the same generator, and are always reported apart.

"What if Authentication-Results is missing or fake?" Missing is recorded as unknown, never as a pass. A fake one written by the sender is ignored: only the headers added by the receiving organisation's own servers are read.

"Why is your F1 so far below the 99% in published papers?" Those numbers are on phishing with links, where the link carries much of the signal. N1 removes that crutch on purpose and measures how far accuracy falls.

"Did you tune anything on the test data?" No. Every rule, pattern, threshold and score number was frozen and fingerprinted before the test split was read, every script refused to run if a fingerprint changed, and every test run is logged. Four scripts were rerun once to fix two reporting bugs, with the reason logged, and no pooled number changed.

"Why no database?" Privacy. The tool reads sensitive company email; storing it would create the breach risk the tool exists to reduce.

"What would you do next?" Test on real data: labelled payload-free business email compromise, real hijacked threads that keep their authentication headers, and a sample of tactics and claims labelled by people. Then widen the claim extractor beyond fixed English phrasings.

16. Glossary

Every term is explained in plain words first. Where it helps, *In our project* says where it shows up, and *Not the same as* heads off a common mix-up.

16.1 Email words

Email (.eml file)

A plain text file: headers, an empty line, then the body.

In our project: the demo/ folder holds made-up .eml files you can load in the analyzer.

Header

One "Name: value" line at the top of an email, such as From or Date. Most headers are written by the sender and can say anything.

Body

The letter itself, below the headers. Can be plain text, HTML or both.

Display name

The free-text name shown next to the address in From, such as "David Chen - Finance Director". Anyone can type anything there.

Domain, registered domain

The part of an address after the @. The registered domain is the part someone bought: mail.acmecorp.com and acmecorp.com share the registered domain acmecorp.com.

Public suffix list

A public list of domain endings (.com, .co.uk, .gov.in) used to find where the registered domain starts.

In our project: used offline through tldextract.

Reply-To

A header that sends replies to a different address from the one in From.

In our project: a Reply-To to another domain is a contradiction only when the body makes a reply-direction claim.

Return-Path, envelope sender

The address bounces go to; the sender recorded on the "envelope" rather than on the letter.

Received line

Added by every mail server an email passes through, at the top. Read from bottom to top, they show the route.

Origin

The first server in the Received chain: where the email entered the mail system.

In our project: part of sending-path drift in N2.

Message-ID, In-Reply-To, References

The unique ID of an email, the ID of the email it answers, and the IDs of the whole chain. Mail programs use them to build threads.

Thread

A conversation: an email and the replies to it.

In our project: rebuilt exactly for the Apache lists and guessed from subjects for Enron.

Quotation, quoted history

Earlier text copied into a reply, often after "On ... wrote:" with lines starting ">".

In our project: N2 checks that quoted text really appeared earlier in the thread.

X-Mailer
: A header naming the mail program that wrote the email.

Mailing list, List-Id
: A server that re-sends one email to all members; List-Id marks such mail. For list mail a different Reply-To and envelope are normal.

MIME
: The standard for emails with several parts (text, HTML, attachments). Parts can be nested inside parts.

SMTP
: The protocol mail servers use to hand emails to each other. On its own it checks nothing about the sender.

DNS, MX record
: The internet's phone book; an MX record says which server accepts mail for a domain.

SPF
: A domain's published list of servers allowed to send for it. Proves the sending server was allowed, not who wrote the email.

DKIM
: A signature on the email made with the sending domain's secret key. Proves that domain signed it and the signed parts were not changed.

DMARC
: Passes when SPF or DKIM passed for a domain that matches the visible From domain; also the domain owner's policy for failures.

  Not the same as: proof of the person. A Gmail account passes DMARC for gmail.com.

Alignment
: Whether the domain that passed SPF or DKIM is the same as the domain in From.

Authentication-Results
: A header in which the receiving server writes its SPF, DKIM and DMARC verdicts.

  In our project: read only from the trusted headers added by the receiving organisation (RFC 8601).

Authenticated domain
: The domain a passing check vouched for.

  In our project: gmail.com for the fake David.

Free mailbox (freemail)
: A provider where anyone can open an address: Gmail, Outlook, Yahoo, Proton.

  In our project: about 100 providers listed by hand, plus open platforms such as onmicrosoft.com.

Lookalike domain
: A domain made to look like a real one: paypa1.com for paypal.com, acme-corp.co for acmecorp.com.

  In our project: scored 0 to 100 after undoing common character tricks.

Punycode
: How domains with non-Latin letters are written in plain ASCII (starting xn--). Attackers use it for lookalikes with Cyrillic letters.

Organisation domain
: The recipient's own company domain.

In our project: taken from the To address or given by the user; without it, internal-affiliation claims are not checkable.

RFC

"Request for Comments": the documents that define internet standards. RFC 7208 (SPF), RFC 6376 (DKIM), RFC 9989 (DMARC) and RFC 8601 (Authentication-Results) are cited in the paper.

16.2 Attack words

Phishing, spear-phishing

Email that tricks you into giving something away, usually through a link; spear-phishing is aimed at one person or company.

Spam

Unwanted bulk mail.

In our project: a category of its own, not counted as an attack.

Ham

Legitimate email: neither spam nor attack.

Advance-fee fraud

"Pay a small fee to receive a large sum."

In our project: the Nigerian fraud collection, the "fraud" category.

Social engineering

Manipulating people rather than computers.

Pretexting

Social engineering built on an invented identity or scenario, where the story is the attack.

Business email compromise (BEC)

Pretexting aimed at company money: fake executives, fake or hacked suppliers, fake payroll changes.

Payload, payload-free

The payload is the part that does technical damage: a malicious link or attachment. A payload-free email has neither.

Impersonation

Pretending to be someone else, through a display name, a free mailbox, a lookalike domain or a forged From.

Spoofing, exact-domain spoof

Writing a false sender. An exact-domain spoof uses the real company's own domain and fails DMARC when the company publishes a policy.

Account takeover

An attacker gets into a real mailbox, usually with a stolen password, and sends from it. Every authentication check passes.

Vendor email compromise

Account takeover at a supplier, used to send fake invoices or new bank details.

Thread hijacking

Replying inside a real, ongoing conversation so the fake message inherits the conversation's trust.

Forged thread

A new email with a fake "earlier conversation" quoted in it.

IBAN

International Bank Account Number. It has a built-in checksum, so many typos can be detected.

In our project: found without regular expressions, checked, and shown masked.

16.3 Persuasion words

Tactic

A pressure technique: authority, urgency, scarcity, reciprocity, social proof, liking, isolation or secrecy.

In our project: detected by DistilBERT; a tactic never creates an alarm, it only makes a contradiction weigh more.

Cialdini's principles

A well-known list of persuasion principles from the psychologist Robert Cialdini; five of our seven tactics come from it.

Urgency versus scarcity

Urgency is time pressure ("by 3 PM today"); scarcity is loss or finality ("your account will be deleted"). One email can have both.

Isolation or secrecy

Cutting the victim off from checking ("keep this between us"). Not one of Cialdini's principles; it comes from fraud reports.

16.4 Machine learning words

Model, classifier

A program that learns patterns from labelled examples and then predicts labels for new ones.

Label

The correct answer attached to an example, such as "this email uses urgency".

Feature

One number the model reads, such as how often a word appears.

Token, tokenizer

A word or piece of a word; the tokenizer cuts text into tokens.

TF-IDF

Turns text into numbers: word counts weighted by how rare each word is.

In our project: paired with logistic regression as the cheap model family in N1.

Logistic regression

A simple model that learns one weight per feature and turns the weighted sum into a probability.

In our project: also used for the four learned comparison systems in N3.

Transformer

A kind of neural network that reads each token in the context of all the others.

BERT, DistilBERT

BERT is a transformer pretrained on a large amount of English. DistilBERT is a smaller, faster version (6 layers, about 66 million learned numbers).

In our project: the tactic classifier and the N1 models.

Pretraining, fine-tuning
: Pretraining teaches a model general language on huge text; fine-tuning trains it a little more on your own labelled examples.

Multi-label
: Several labels can be true at once; the model gives one probability per label.

Threshold
: The cut above which a probability counts as "yes".

  In our project: one per tactic, chosen on validation: authority 0.55, urgency 0.45, scarcity 0.65, secrecy 0.90.

Epoch, seed
: An epoch is one pass through the training data. A seed fixes the randomness so a run can be repeated.

GPU, Colab
: A GPU is a processor that trains neural networks fast; Google Colab rents one for free in the browser.

Weights, safetensors
: Weights are the learned numbers of a model. safetensors is a file format that holds only numbers, so loading it cannot run hidden code.

Half precision (fp16)
: Storing numbers with less detail to train faster; results differ very slightly from full precision.

Rule-based
: Decided by hand-written rules instead of learning.

  In our project: the claim extractor, the verifiers and the score.

Named-entity recognition, spaCy
: Marking names of people and organisations in text; spaCy is a library that does this.

Regular expression, ReDoS
: A regular expression is a pattern over characters. ReDoS is an attack where a crafted input makes a badly written pattern run for hours.

Token pattern
: A pattern over tokens, such as "this is [person] from [department]". Its cost grows only with text length.

  In our project: 85 of them in src/claims/patterns.py.

LIME
: Explains one prediction by hiding random words many times and fitting a small model to how the prediction changes.

  In our project: highlights the words behind each tactic; written by hand.

Large language model (LLM)
: A very large model trained on text to predict the next word, such as Gemini or GPT-4.

  In our project: used only offline, to label emails and write test cases; never at run time.

Annotator, tie-breaker
: Whoever gives labels. Here two language models labelled separately and a third decided where they disagreed.

Temperature
: A setting that controls how random a language model's answers are; 0 makes them as repeatable as possible.

Prompt injection
Text inside the data that tries to give a language model new instructions ("ignore your instructions").

16.5 Evaluation words

Train, validation, test
Learn on train; choose settings on validation; measure once on test.

Leakage, data snooping
Information from the test data influencing a choice, which makes the test score look better than it is.

Grouped split
Keeping related items (one conversation, one campaign, one synthetic pair) in the same split.

Confusion matrix
The four counts: true positives, false positives, false negatives, true negatives.

Precision, recall, F1
Precision: of the flagged, how many were attacks. Recall: of the attacks, how many were flagged. F1 combines both.

Macro-F1
The plain average of several F1 scores, one per class.

False alarm rate (false positive rate)
The share of legitimate emails wrongly flagged.

AUC
The chance that a random attack scores higher than a random legitimate email; 0.5 is chance, 1.0 is perfect. Needs no cut.

Cut, operating point
The score above which an email is flagged.
In our project: the learned systems were held to the routed score's validation false alarm rate (0.14%), with a 0.5% floor.

Confidence interval
A range of plausible values for a measured number, here 95% intervals.

Bootstrap
Getting an interval by resampling the test data many times (1,000 here) and recomputing the number each time.

Paired difference
The difference between two systems computed on the same resamples. If its interval includes 0, neither system may be called better.

Wilson interval
A formula for the interval of a proportion that behaves well with small counts.
In our project: used for the false-alarm budget in Phase 10.

Count-only cell
A result with fewer than 10 positives, reported as raw counts instead of a rate.

Baseline
The simple thing a system is compared against.
In our project: the keyword lists for tactics; the four learned systems for detection.

Ablation
Removing or swapping one component and measuring again.

Corpus labels
Labels that come from the collection itself (this collection is phishing), not from anyone reading each email.
In our project: the N1 labels.

Confound, shortcut
A confound is a hidden difference between groups that a model can use instead of the real signal; a shortcut is the model using it.
In our project: links and collection style.

Style-confound test
Measures how well wording alone can tell two collections apart.

Hijack benchmark
Real threads with one synthetic message put in place of a real reply, in three attack variants and two honest negatives.

Freeze record
A file of fingerprints (SHA-256 hashes) of every frozen component, committed before the test; every test script checks it.

Hash (SHA-256)
A fingerprint of a file or text: any change produces a completely different value.

Dress rehearsal
Running every evaluation script on validation before the test, to find bugs in the evaluation code safely.

Test log, rerun
results/eval_test_log.csv records every test run. A rerun is allowed only to fix a bug in an evaluation script, with the reason logged.

Paraphrase test
Rewording attack emails with a language model to see whether the detection survives.

16.6 Software words

Repository (repo), Git, GitHub
A project folder with its full history; Git records the history on your laptop; GitHub hosts a copy online.
In our project: github.com/Nagasai-Datta/PretextGuard.

Commit, push, commit hash
A commit saves a named version; push uploads it; the hash (such as 18cec7f) names one version.

venv, requirements.txt
A private Python environment for one project (like node_modules) and the exact list of its libraries.

.env
A file with secrets (API keys), never committed.

API, endpoint, route
A set of addresses a program can send requests to; one address is an endpoint or route, such as POST /analyze.

FastAPI, Pydantic, uvicorn
Python's equivalent of Express; a schema library (like Zod); the server that runs the app.

API key
A secret that must accompany a request.
In our project: sent in the X-API-Key header and added by the website's proxy.

Rate limit
A cap on how many requests a client may send per minute.

CORS
A browser rule about which websites may call an API.
In our project: open to any origin on purpose (CORS_ALLOW_ANY_ORIGIN = True), with no credentials and the key still required.

XSS (cross-site scripting)
Getting a browser to run an attacker's script by sneaking it into displayed text.
In our project: prevented by drawing every string as plain text.

Content-Security-Policy
A response header telling the browser what a page may load and connect to.

Proxy
A server that forwards requests, here adding the API key.

React, Vite, Tailwind, Recharts
The UI library, the build tool and dev server, the styling system and the chart library of the website.

Parquet
A compressed table file format that keeps column types.

Self-test, mutation check
A self-test runs the code on hand-made inputs and checks the outputs. A mutation check breaks something on purpose to prove the tests notice.

Audit log
A record of events with a fixed list of fields and no email content.

16.7 Research and paper words

Conference paper, peer review, double-blind
A short paper judged by other researchers before acceptance; in double-blind review neither side knows the other's identity.

IEEE
A large engineering association whose conference template and reference style the paper follows.

Novelty, contribution
What a paper adds that did not exist before.

Related work
What others have done and how the new work differs.

Citation, reference, source table
A citation points to a reference such as [7]; the source table lists every reference with how it was checked and what it may support.

Preprint, arXiv
A paper posted publicly before or without formal review; arXiv is the main free site for them.

Threat to validity, limitation
A reason a result might not mean what it seems to; something the work does not do, stated openly.

16.8 The project's own words

PretextGuard
The name of the project and the repository.
Not the same as: the name in the paper, which is "the claim-routed verification pipeline".

Claim
A statement an email makes about itself, of one of eleven types.

Claim type
affiliation_internal, affiliation_external, authority, reply_direction, signature_contact, prior_relationship, payment_request, payment_change, credential_request, gift_card, data_request.

Confidence (of a claim)
0.9 for a strong pattern, 0.6 for a weak one. The strength of the rule, not a probability.

Router, routing table
Sends each claim to the verifier that can test it.

Verifier
A set of rules that checks claims: header verifier (N3), request verifier, thread verifier (N2).

Rule, rule name
One check inside a verifier, named hv_, rv_ or tv_ plus a description, such as hv_int_freemail.

Ledger, ledger row
The record of every check: claim, verifier, rule, evidence, verdict, severity and reason.

Contradiction, consistent, not checkable
The three verdicts: the evidence disagrees; it fits (or nothing contradicts it); it is missing.

Severity
High, medium or low: the strength of the rule that found a contradiction, not a probability.

Reliability factor
1, 0.5 or 0 per rule, set from how often the rule fired on legitimate mail in the train split.

Risk score, band
0 to 100 from the ledger; Low risk (0 to 34), Suspicious (35 to 69), High risk (70 to 100).

Coverage
How many claims were found, checked, contradicted, consistent and not checkable.

False-alarm budget
The declared limit: at most 1% High risk and 5% Suspicious or above among checked legitimate emails per source.

Conditioning on the claim
Reading the same header evidence differently depending on what the email claims.

Tactic onset, request drift, sending-path drift, thread integrity
The four thread signals of N2 (Section 4.7).

Flip point
The first message in a thread with a medium or high contradiction.

Variant A, B, C
: The hijack benchmark's attacks: A account takeover, B lookalike domain swap, C forged thread.

Benign twin
: A synthetic legitimate email written from the same template as a synthetic attack, to control for writing style.

Model A, model B
: The N1 models: A trained on raw bodies, B on redacted bodies.

Text only, headers only, parallel fusion, flat classifier
: The four learned comparison systems of the N3 and architecture experiments.

analyze()
: The function in src/router/pipeline.py that runs the whole pipeline for one email or thread and returns the report.

Master document, context file
: docs/master_document.md is the single source of truth; docs/PretextGuard_Context.md is the handover file every new chat starts from.

Phase
: One stage of the build, 0 to 13; Phase 14 was cancelled when the paper became the evaluated deliverable.